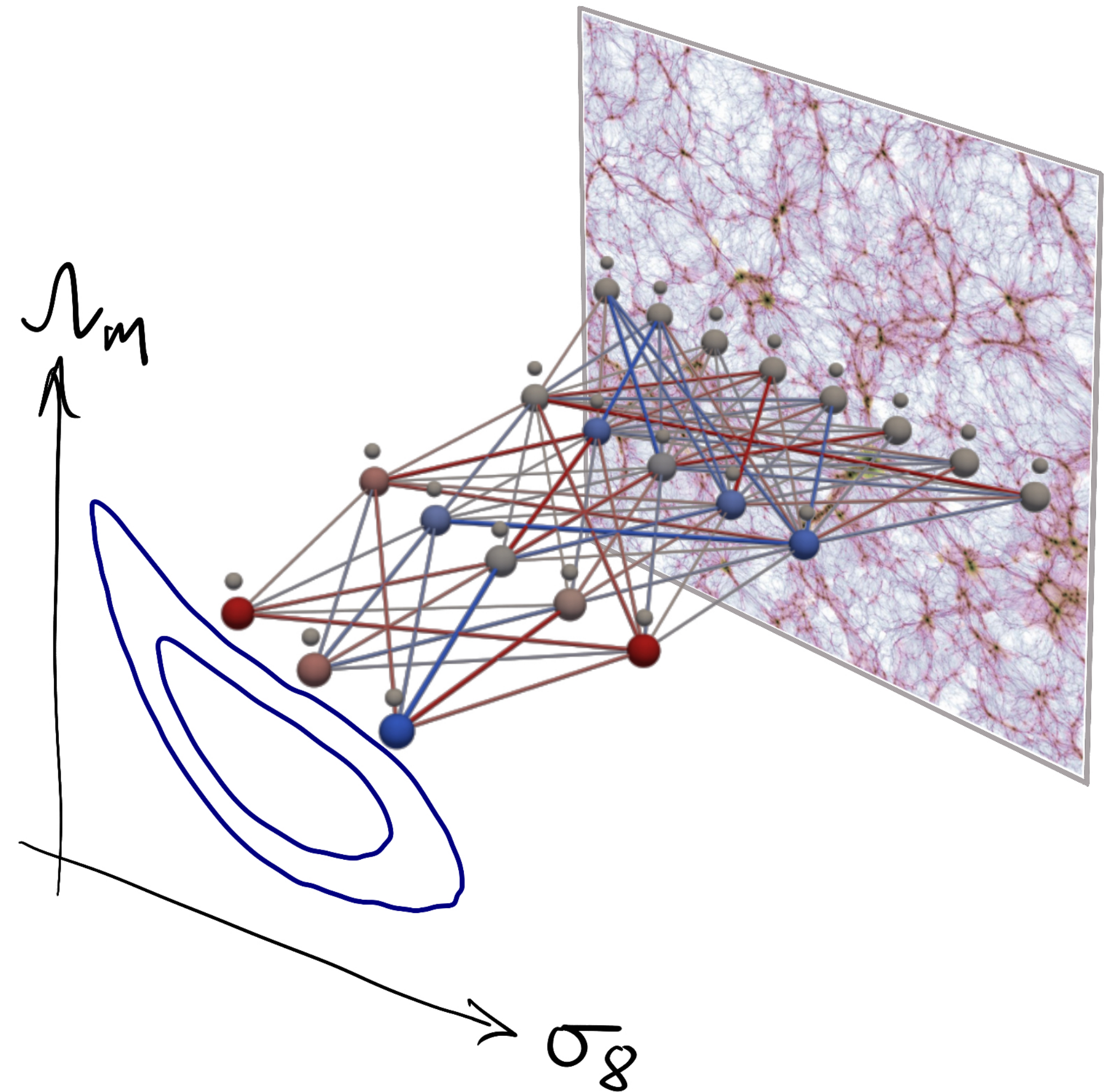


# Simulation-based inference

Jed Homer LMU/ODSL



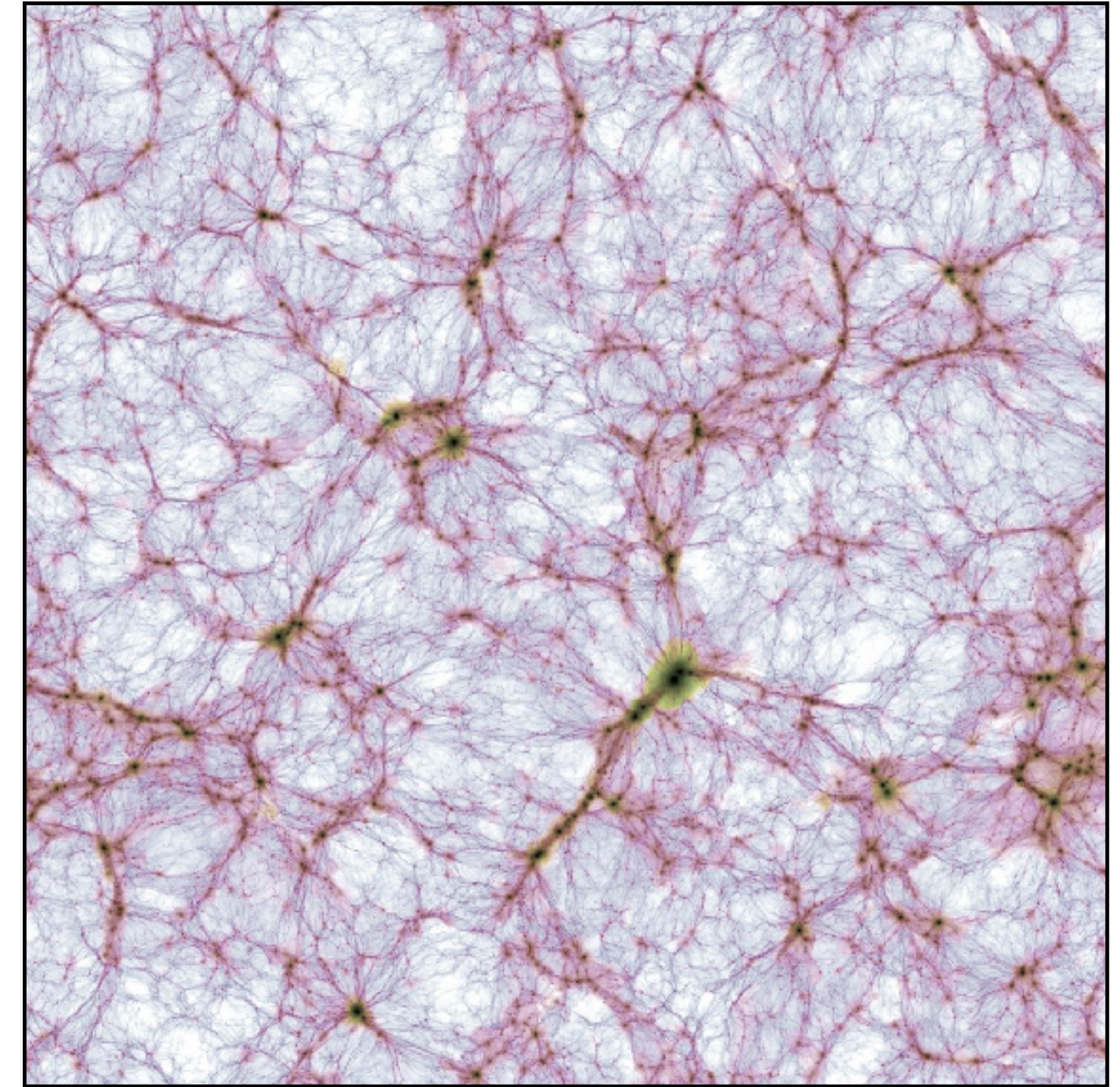
# Machine learning based inference = SBI

- What is it?
- Why?
- Normalising flows
- Compression with neural networks or linear methods
- Does it work?

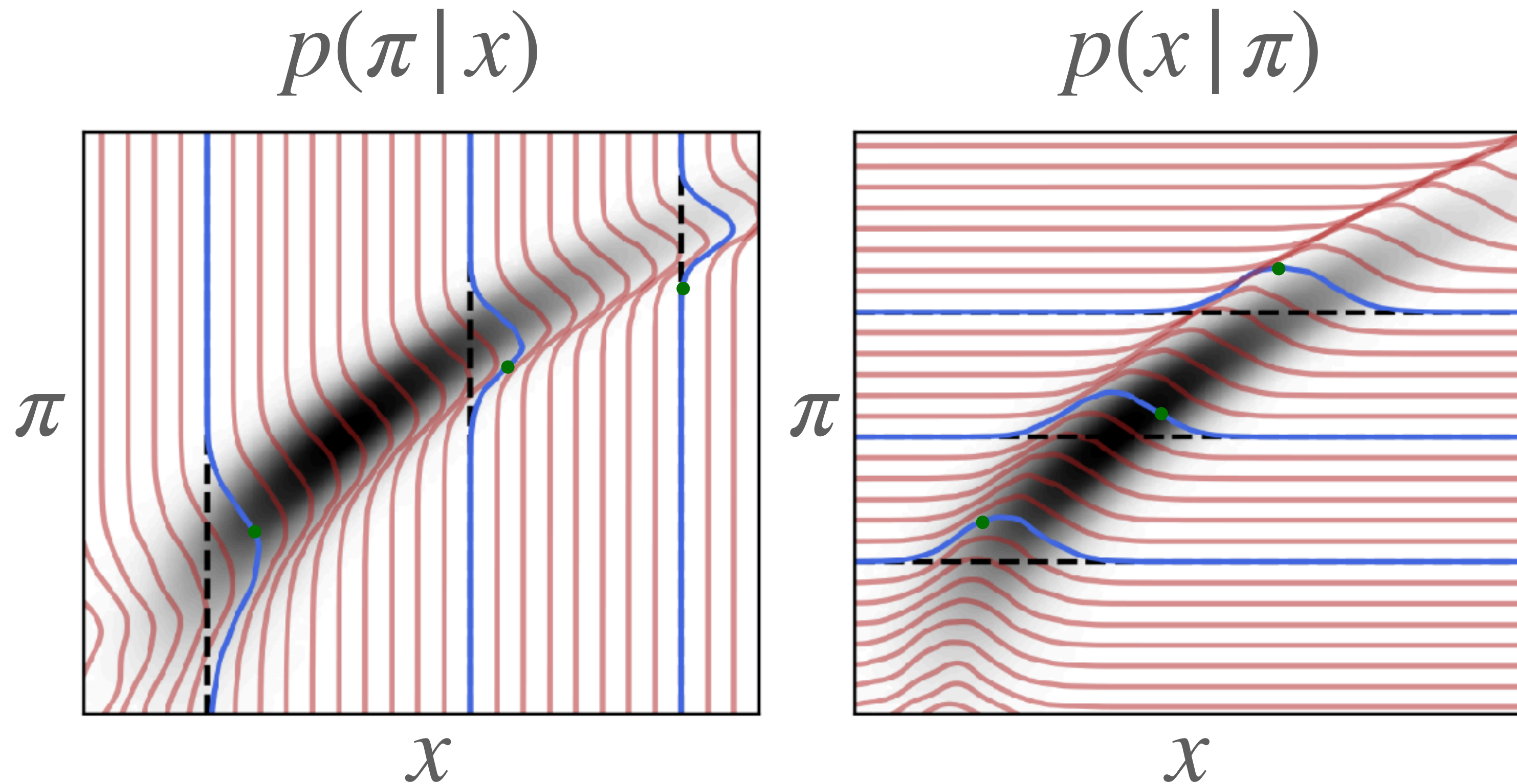
# Why simulation-based inference (SBI)?

$$p(\pi | \hat{x}) \propto p(\hat{x} | x[\pi], \pi) p(\pi)$$

- **Likelihood**  $p(\hat{x} | \pi)$  may be non-Gaussian
- **Expectation**  $x[\pi]$  is a complex non-linear function of  $\pi$
- **Covariance**  $\Sigma$  may depend on  $\pi$
- $n_x$  is increasing with new surveys, inversion of  $\hat{\Sigma}$

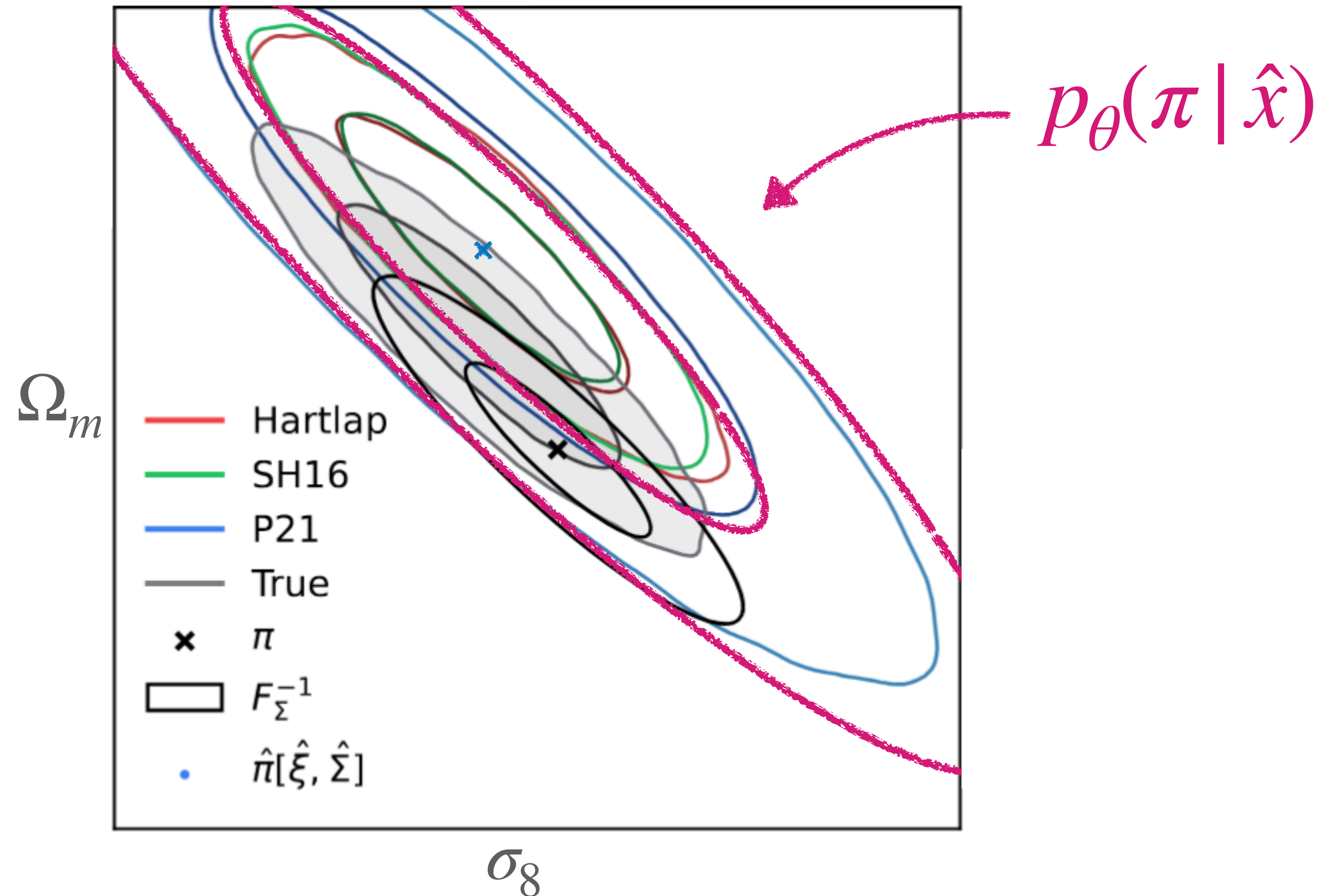


# Simulation-based inference (SBI)

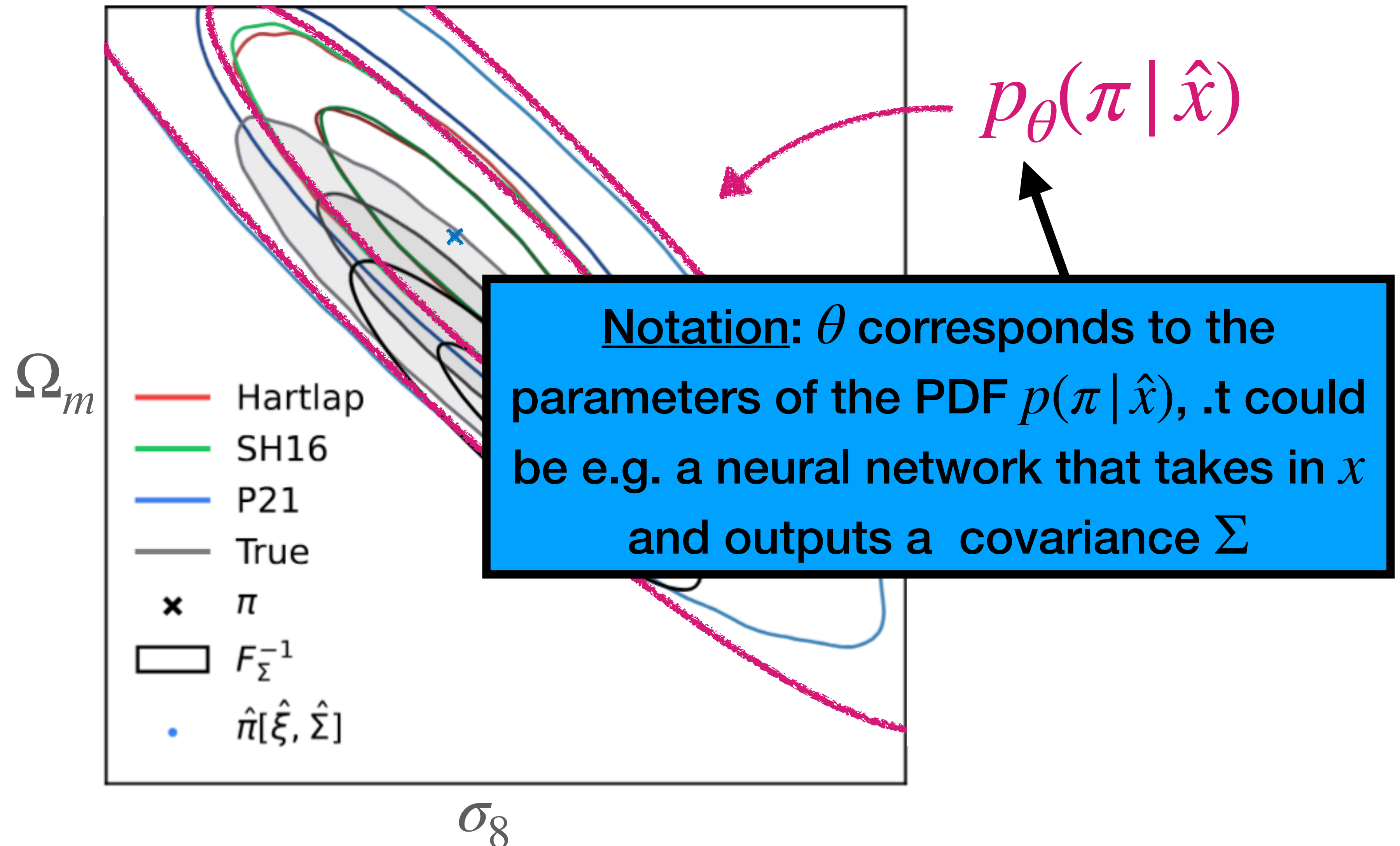


- **Analytic methods:** model PDF  $p(x | \pi)$  across space  $(x, \pi)$
- **SBI:** fit the expectation  $x[\pi]$ , covariance  $\Sigma$  and PDF  $p(x | \pi)$  from simulations  $(x, \pi)$

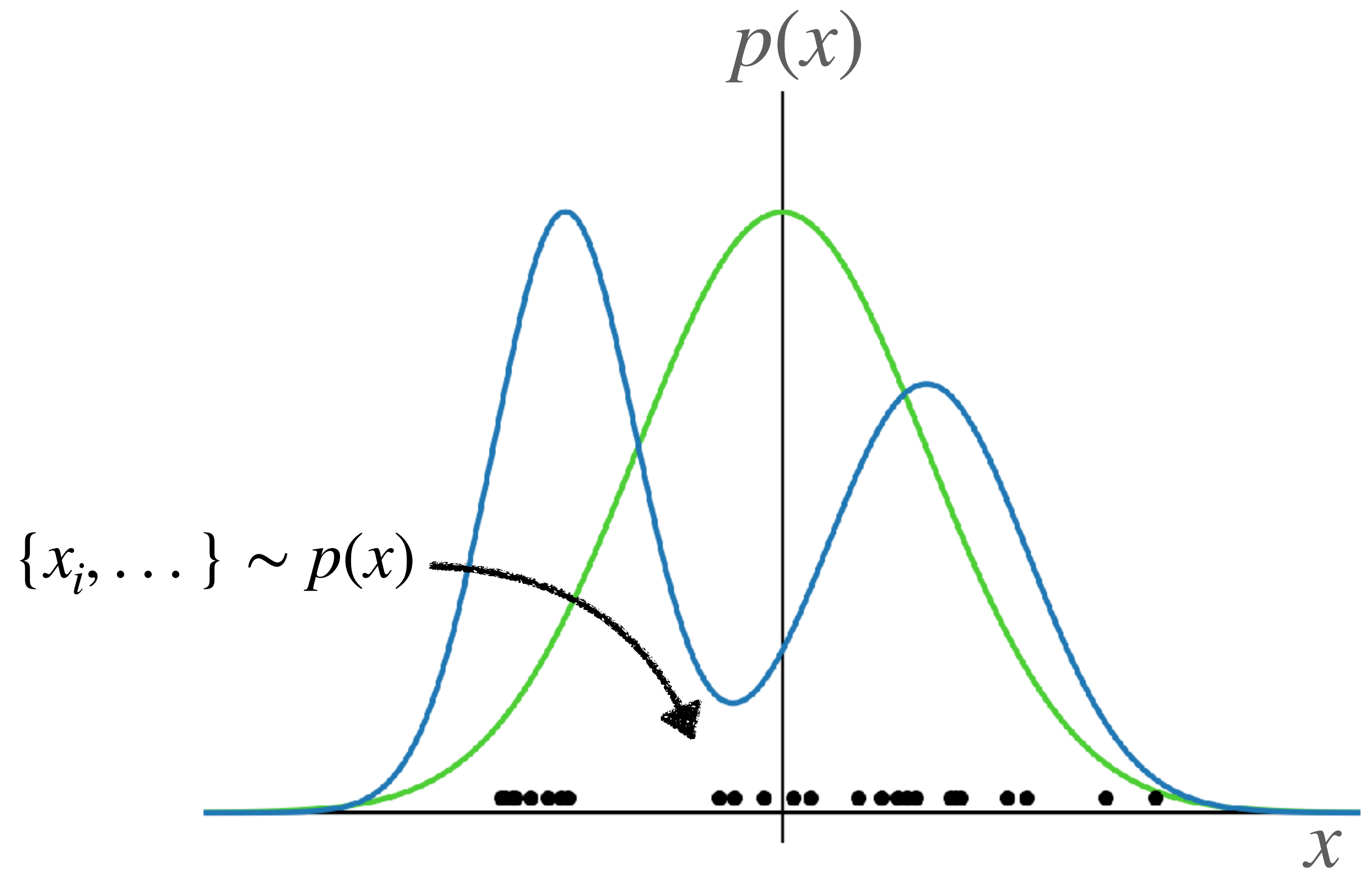
# What contour does SBI draw?



# What contour does SBI draw?



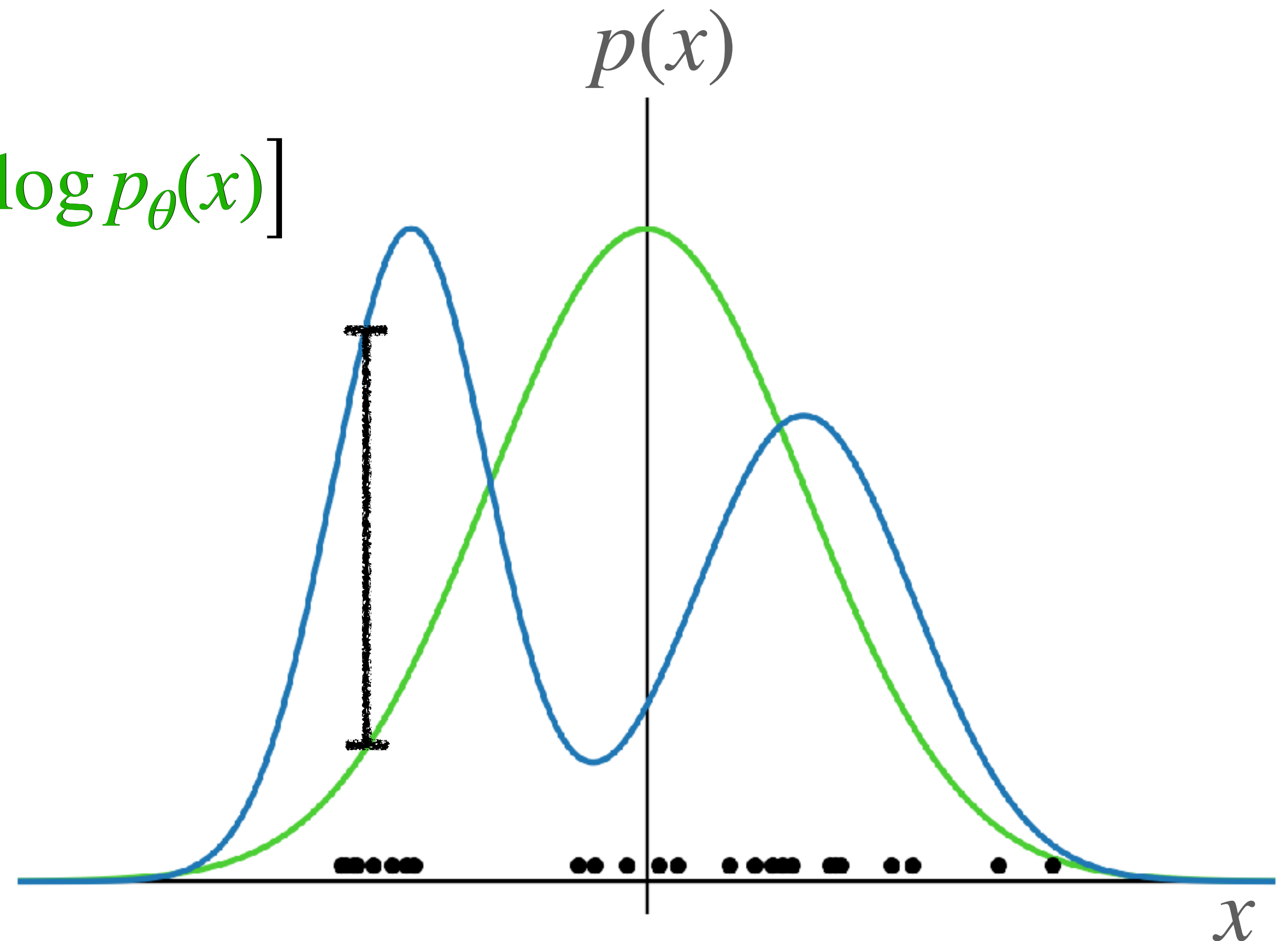
# Fitting $p_{\theta}(x \mid \pi)$ without knowledge of $p(x \mid \pi)$



# Fitting $p_\theta(x \mid \pi)$ without knowledge of $p(x \mid \pi)$

- **KL-divergence**  $\mathcal{D}_{KL}$ : expected information difference between two PDFs, across all  $x$

$$\mathcal{D}_{KL}[p_\theta(x) \parallel p(x)] = \mathbb{E}_x[\log p(x) - \log p_\theta(x)]$$

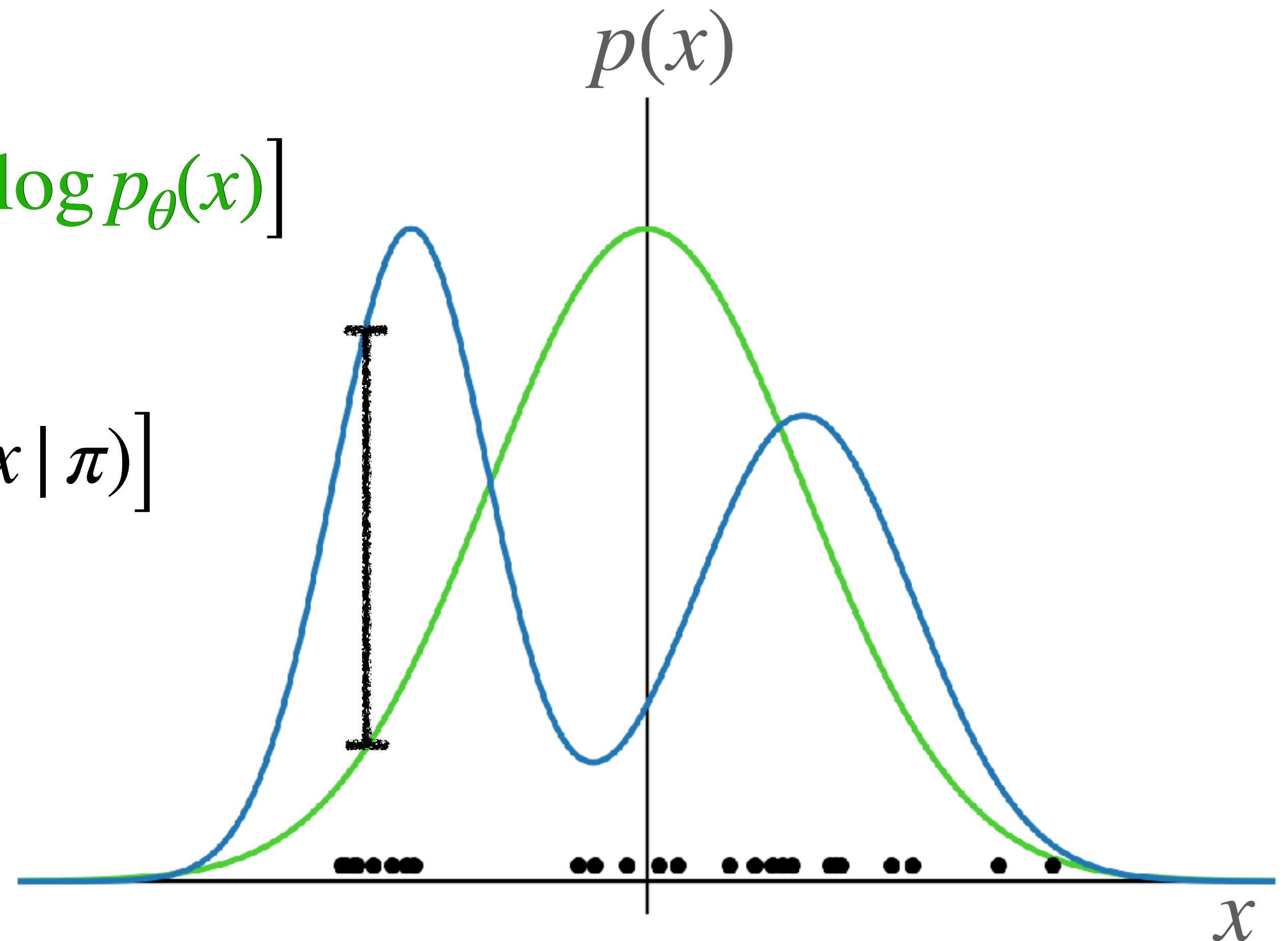


# Fitting $p_\theta(x \mid \pi)$ without knowledge of $p(x \mid \pi)$

- **KL-divergence**  $\mathcal{D}_{KL}$ : expected information difference between two PDFs, across all  $x$

$$\mathcal{D}_{KL}[p_\theta(x) \parallel p(x)] = \mathbb{E}_x[\log p(x) - \log p_\theta(x)]$$

$$\Rightarrow \mathcal{L}(x, \pi, \theta) = -\mathbb{E}_{\pi, x}[\log p_\theta(x \mid \pi)]$$



# Maximum likelihood with $p_{\theta}(x) \leftrightarrow$ Minimisation of $\mathcal{D}_{\text{KL}}$

- **We don't need to know  $p(x)$  to fit  $p_{\theta}(x)$  like in supervised learning...**

$$\mathbb{E}_{\pi, x} [\log p_{\theta}(x | \pi)] \approx \frac{1}{N} \sum_i \log p_{\theta}(x_i | \pi_i)$$

...we only **need samples  $x \sim p(x)$  and a  $p_{\theta}(x)$**  that can be evaluated.

# Remember from lecture 1: **Change-of-variables**

- Express PDF  $p(x)$  in terms of another PDF  $p(z)$

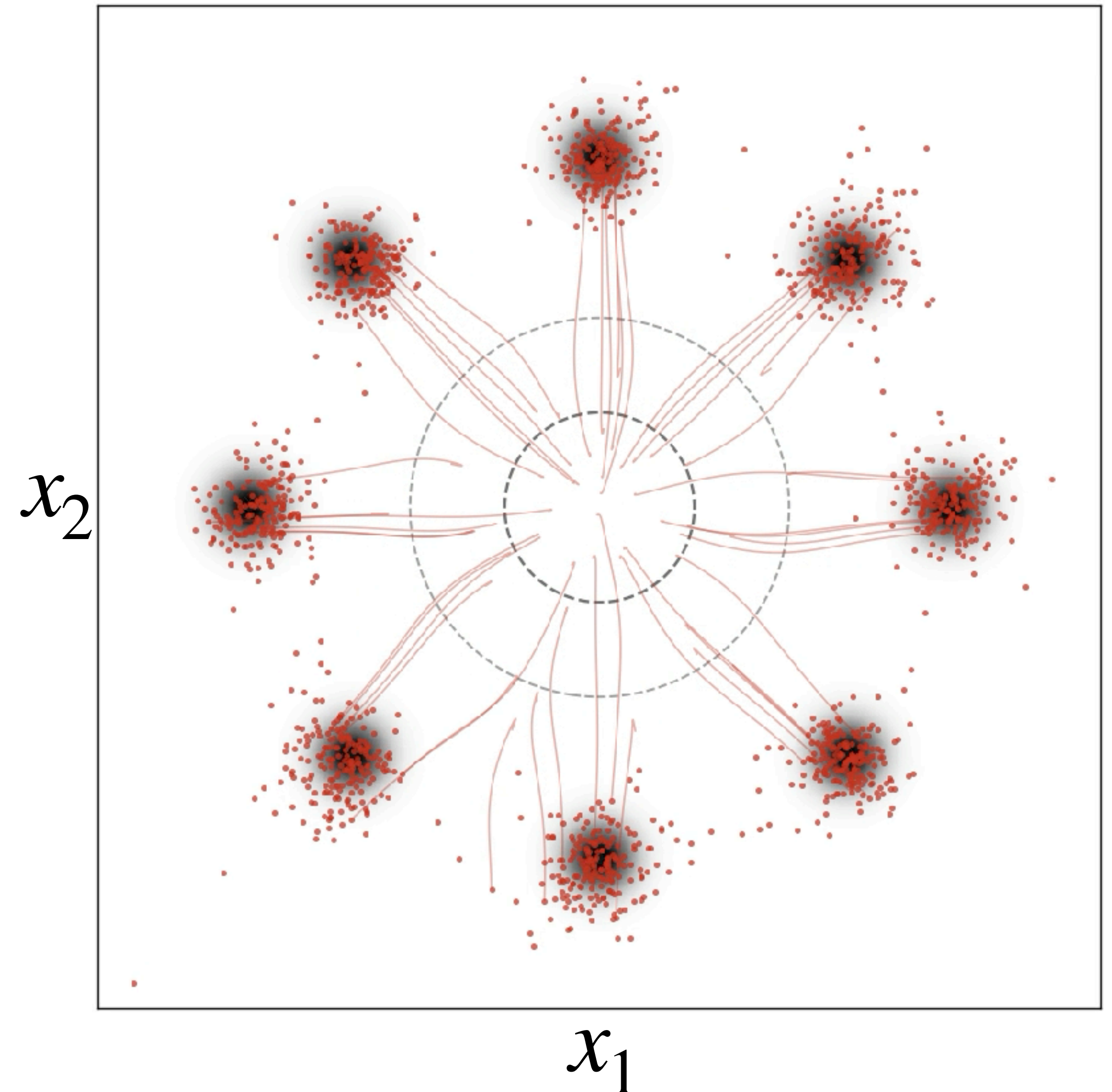
$$p(x) = |J_f(x)| \cdot p(z)$$

# Remember from lecture 1: **Change-of-variables**

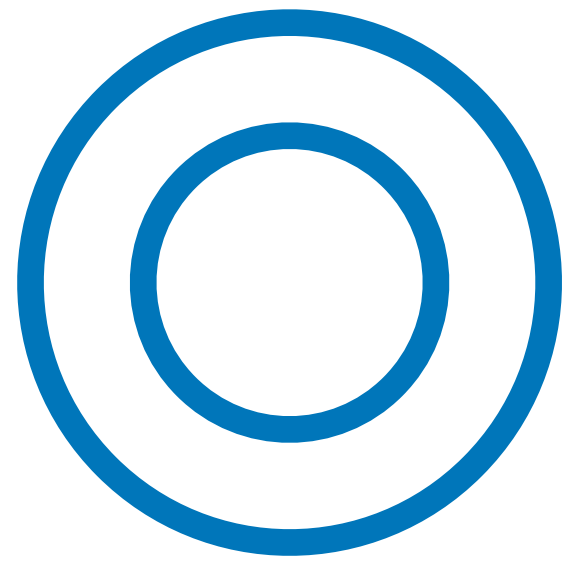
- Express PDF  $p(x)$  in terms of another PDF  $p(z)$

$$p(x) = |J_f(x)| \cdot p(z)$$

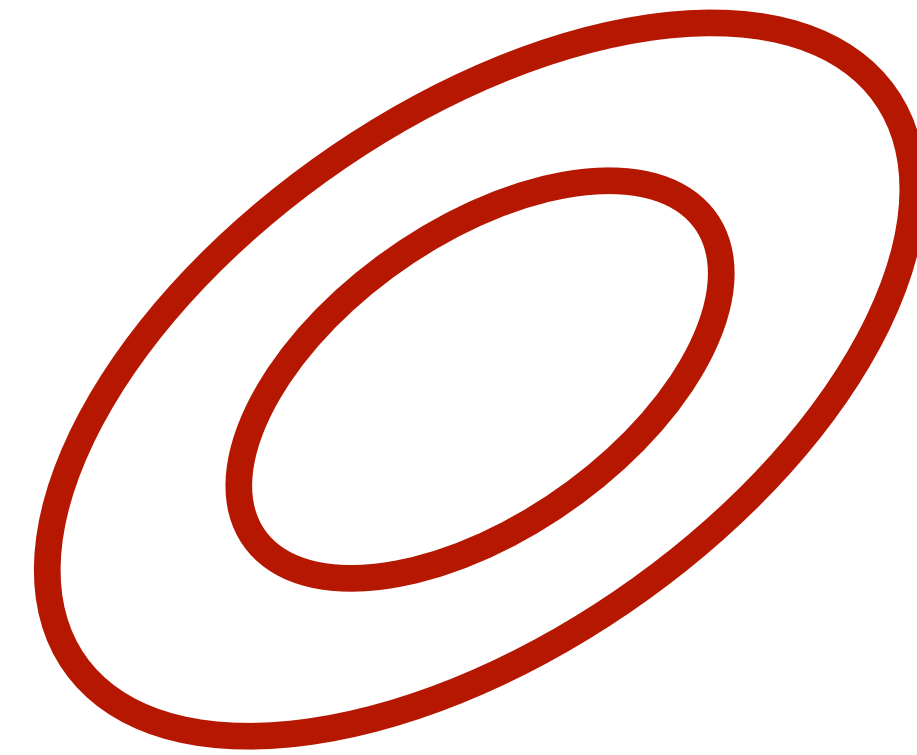
- Basis of many ML models, used for analysing data we cannot analytically model.



# Analytic example



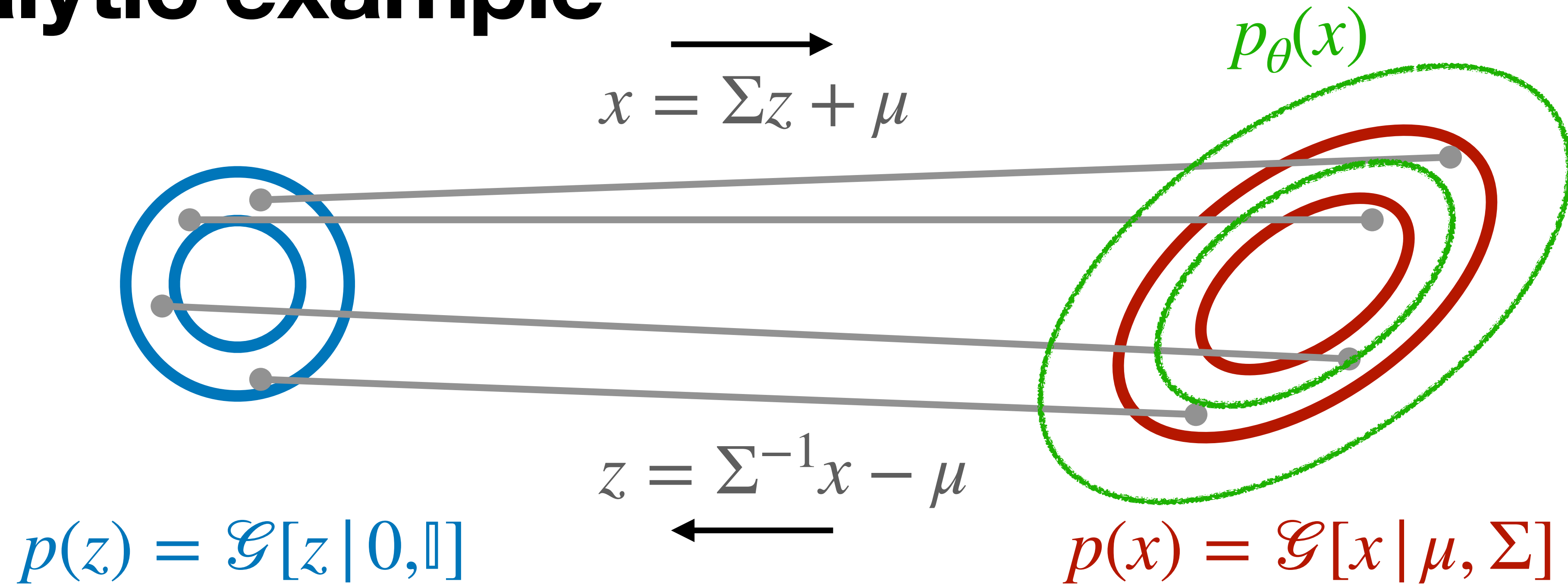
$$p(z) = \mathcal{G}[z \mid 0, \mathbb{I}]$$



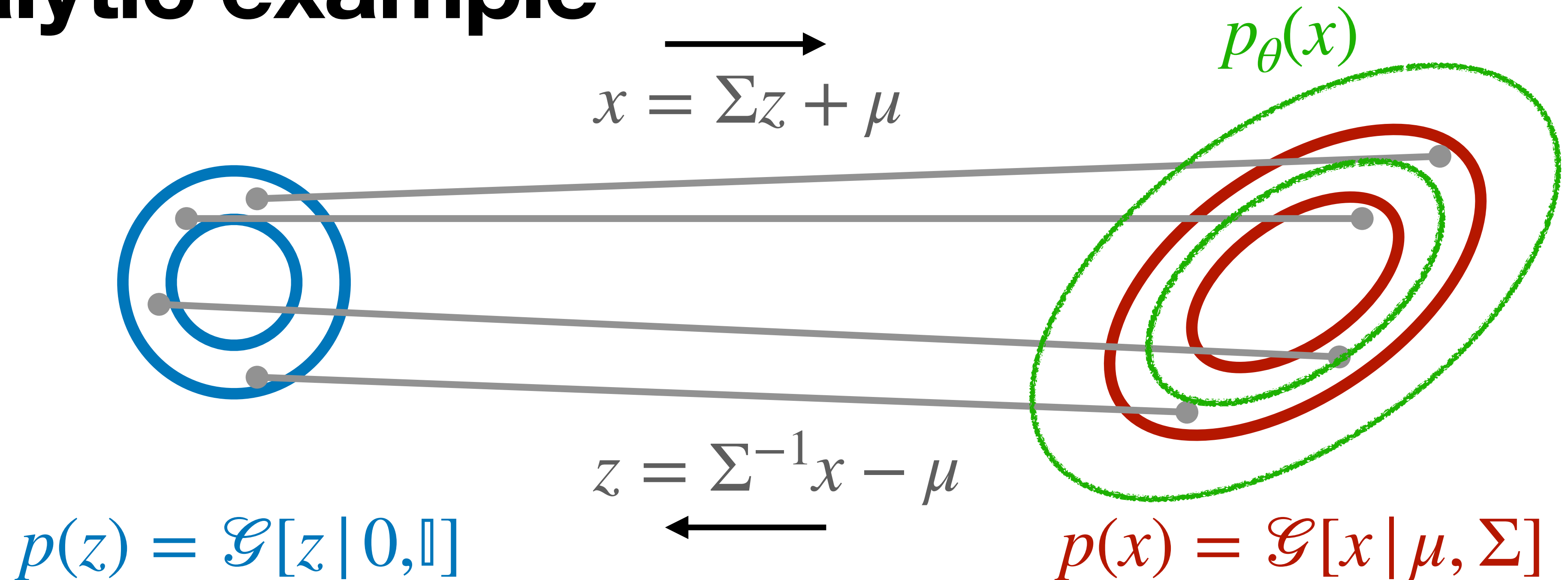
$$p(x) = \mathcal{G}[x \mid \mu, \Sigma]$$

- First we calculate a transform between  $p(z)$  and  $p(x)$ ,
- Then we show a neural network how to do it using only  $\{x_i, \dots\} \sim p(x)$

# Analytic example



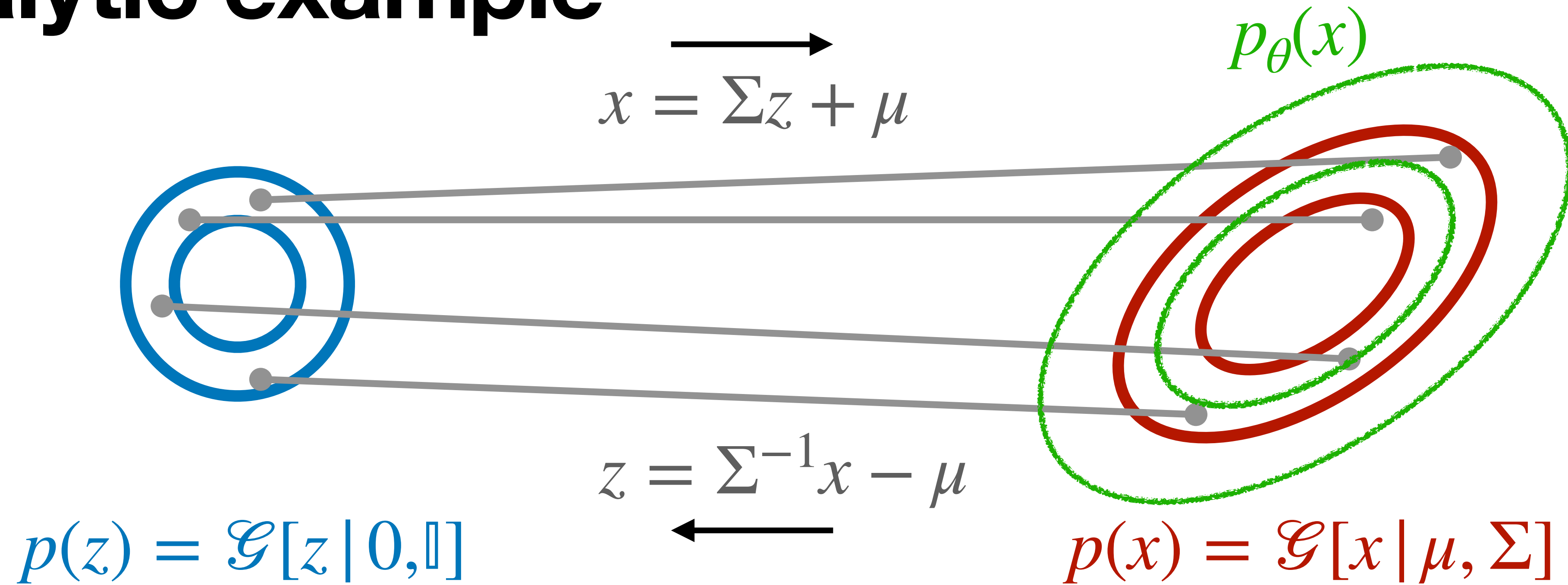
# Analytic example



$$p_\theta(x) = |J[f_\theta](x)| \cdot p(f_\theta(x))$$

$f_\theta(x) = \Sigma_\theta^{-1}x - \mu_\theta$

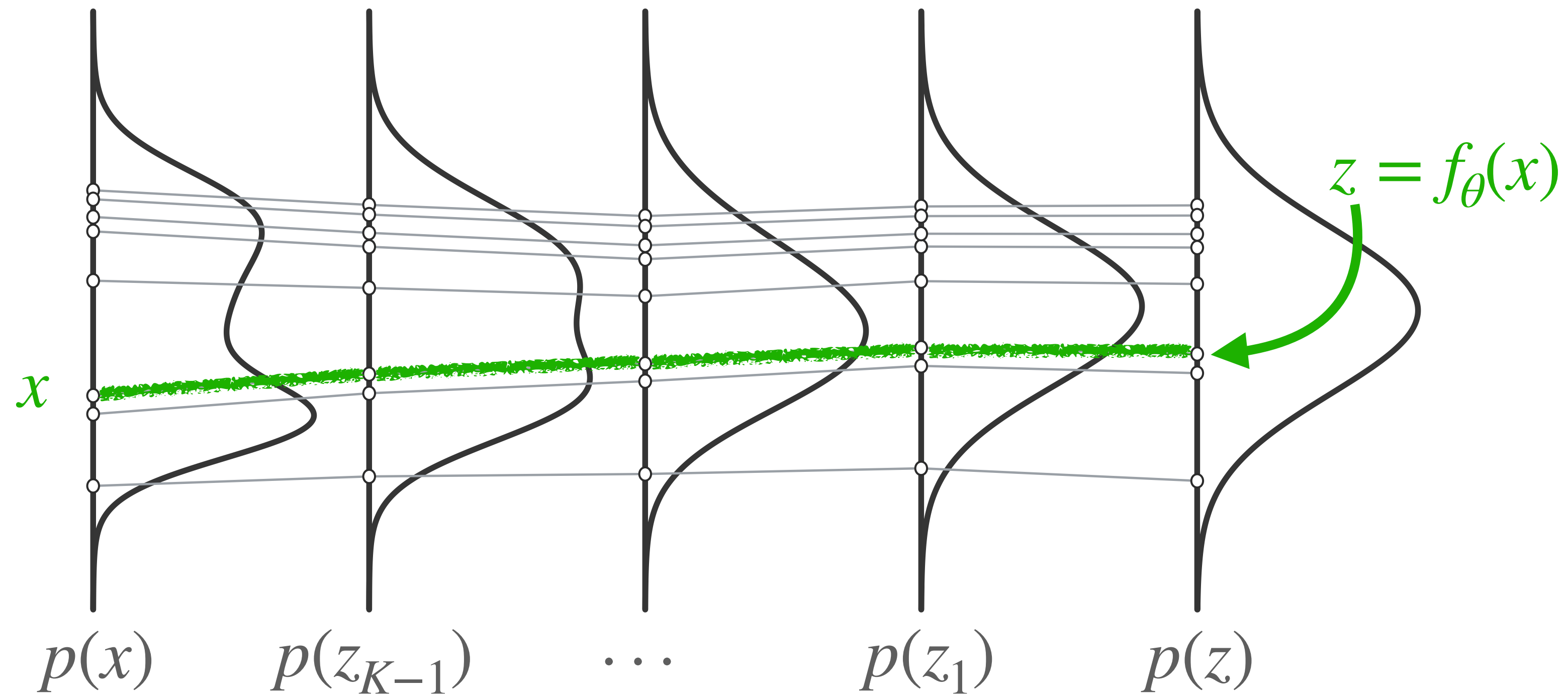
# Analytic example



$$\Rightarrow \log p_\theta(x) = \log p(z) - \log |\Sigma_\theta^{-1}| + \text{const.}$$

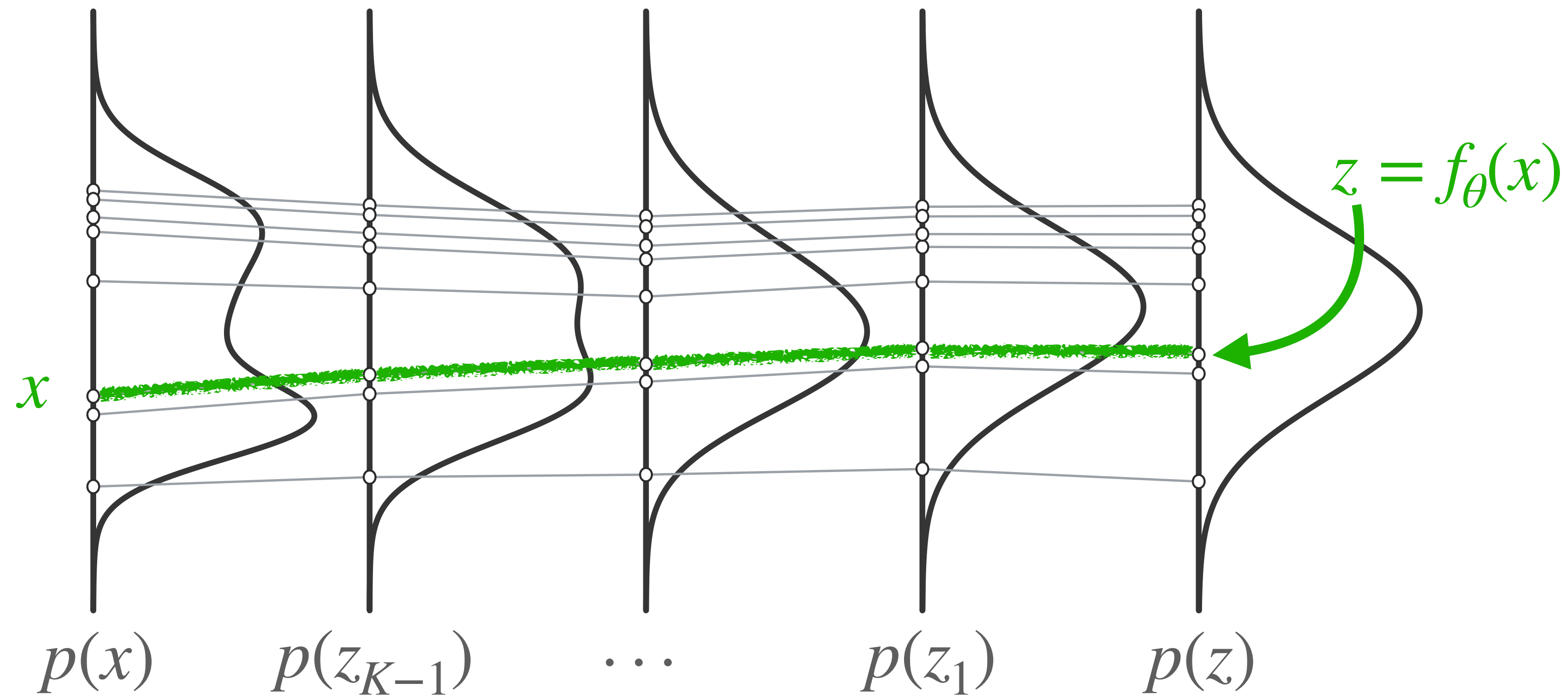
# Normalising Flows

- In practice, a normalising flow requires  $K$  separate transforms, for complex  $p(x)$



# Normalising Flows

- In practice, a normalising flow requires  $K$  separate transforms, for complex  $p(x)$

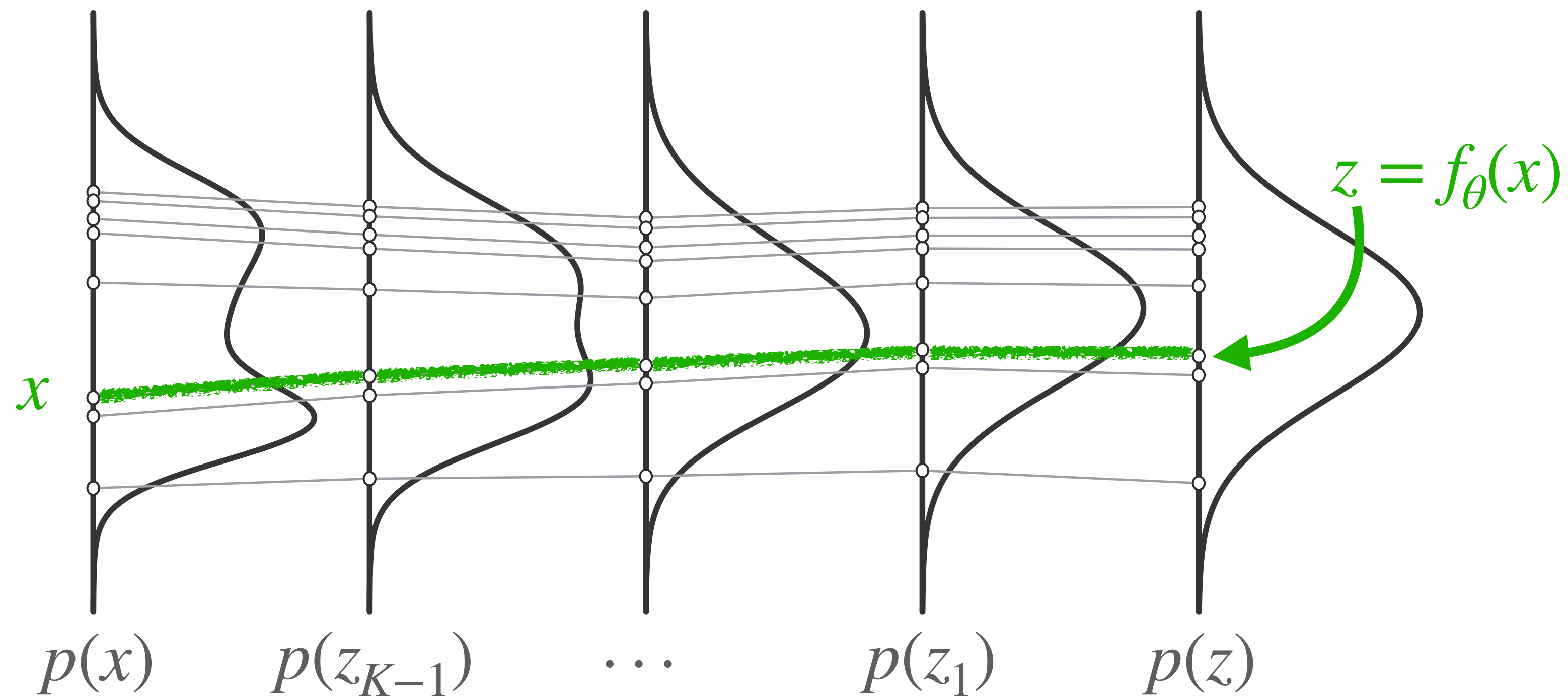


$$p(x) = p(z) \cdot |\det \mathbf{J}[z](x)|$$

$$= p(f_\theta(x)) \cdot |\det \mathbf{J}[f_\theta](x)|$$

# Normalising Flows

- In practice, a normalising flow requires  $K$  separate transforms, for complex  $p(x)$



$$p(x) = p(z) \cdot |\det \mathbf{J}[z](x)|$$

...in the case the model  $f_\phi$  has multiple transforms

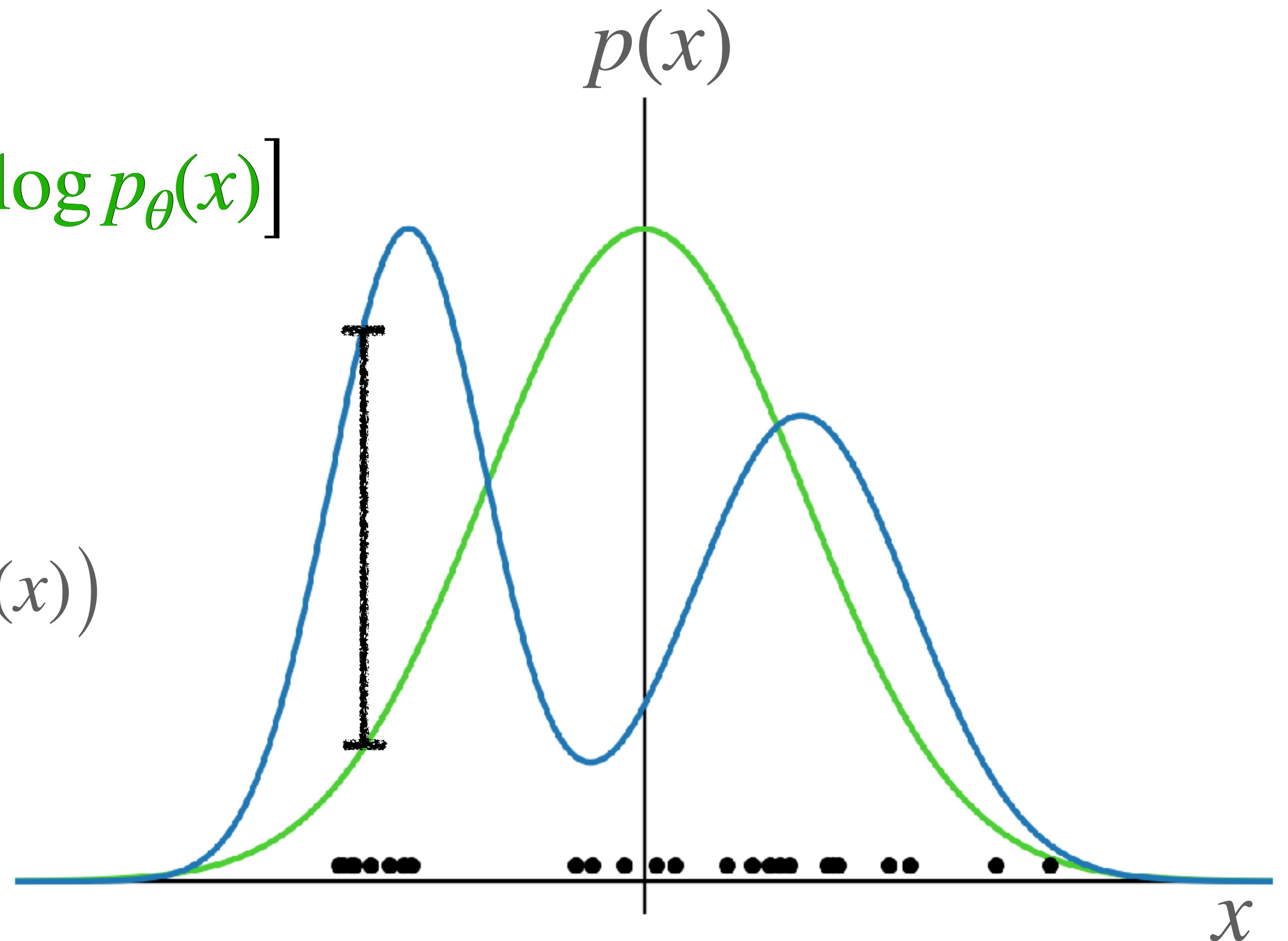
$$= p(f_\theta(x)) \cdot |\det \mathbf{J}[f_\theta](x)| = p(f_{\theta_{1:K}}(x)) \cdot \prod_{k=1}^K |\det \mathbf{J}[f_\theta](x_{k-1})|$$

# Fitting the normalising flow PDF $p_\theta(x)$

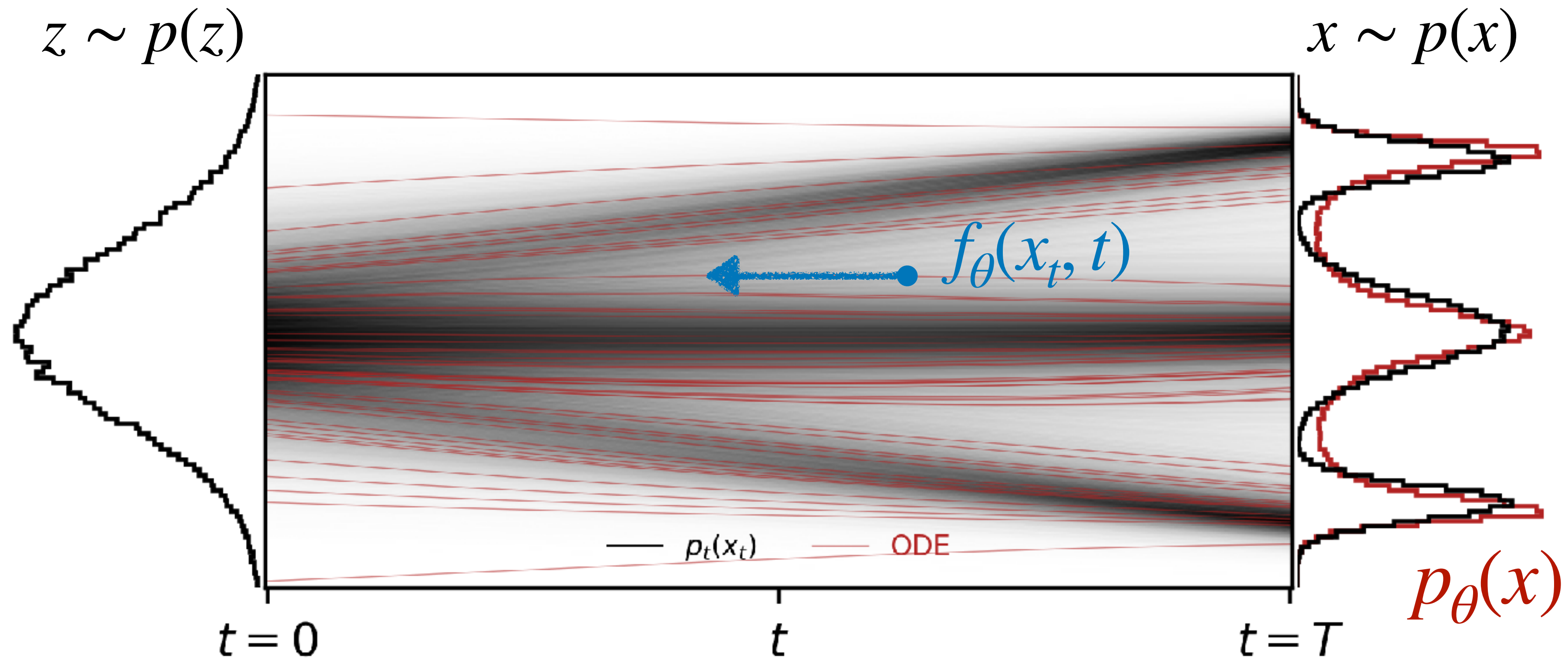
- **KL-divergence**  $\mathcal{D}_{KL}$ : expected information difference between two PDFs, across all  $x$

$$\mathcal{D}_{KL}[p_\theta(x) \| p(x)] = \mathbb{E}_x[\log p(x) - \log p_\theta(x)]$$

$$\log p_\theta(x) = \log \left| \det \frac{df_\theta}{dx} \right| + \log p(f_\theta(x))$$

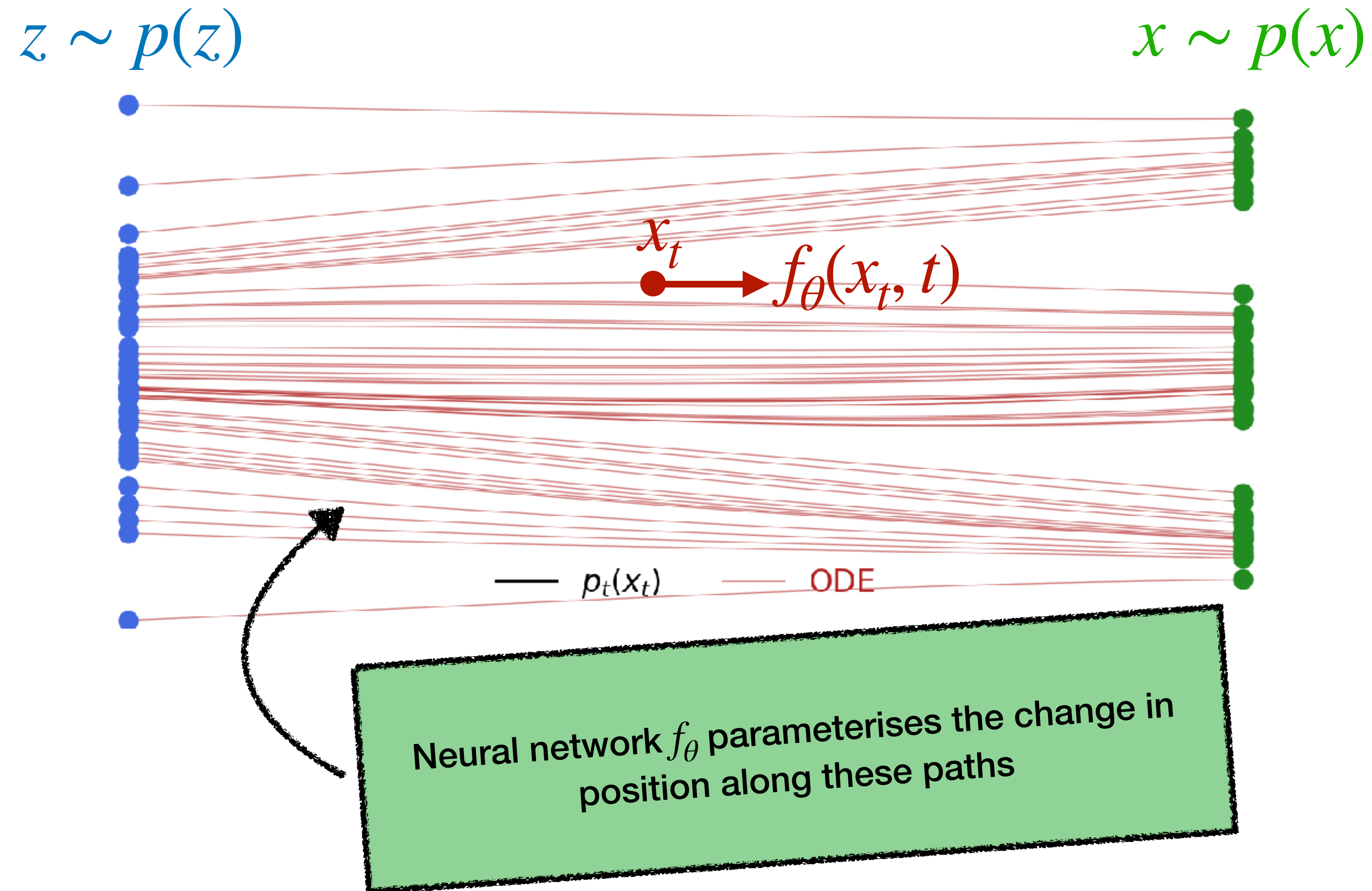


# Continuous normalising flows

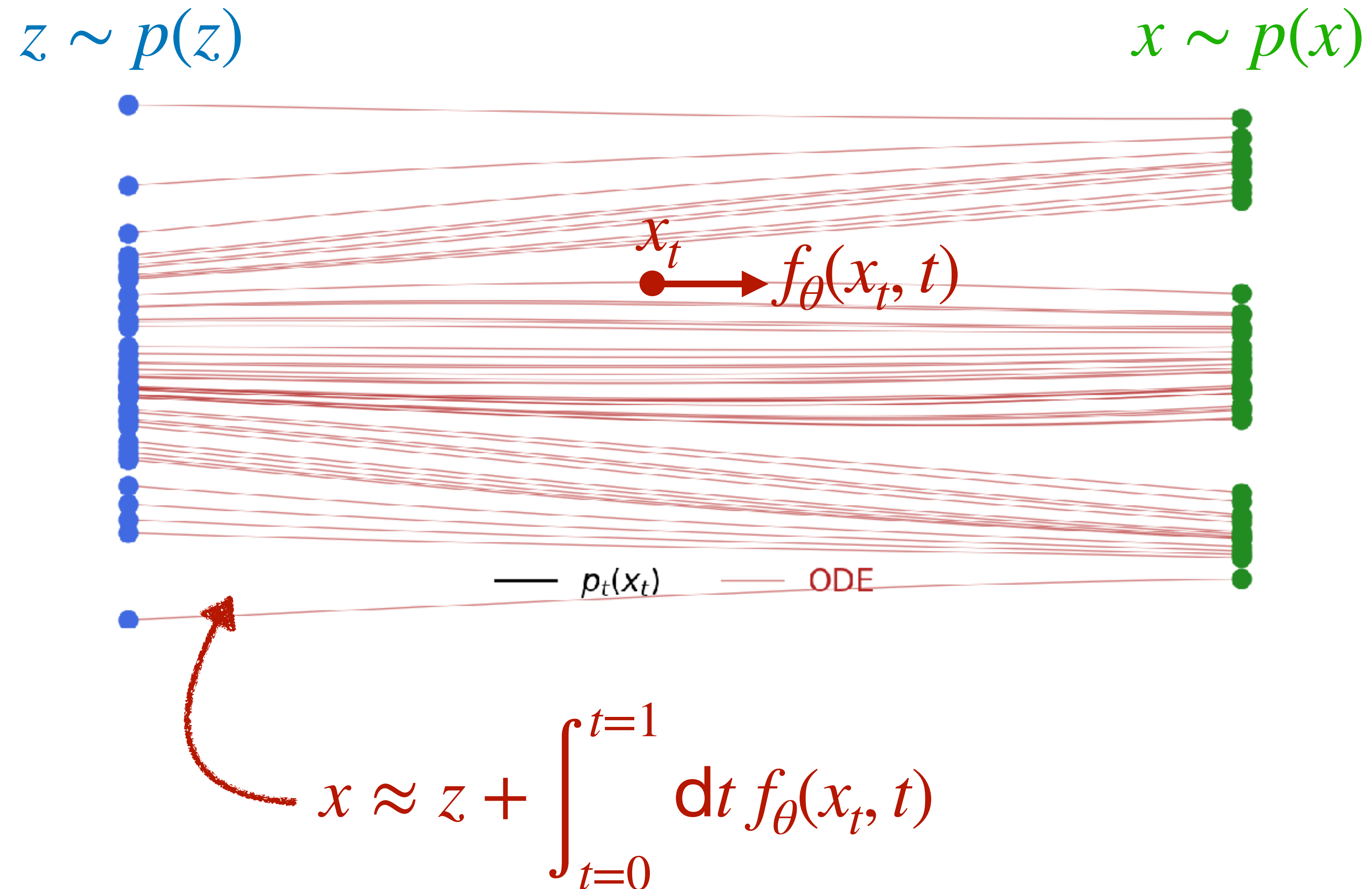


$$\log p_\theta(x) = \log p(z) + \int_{t=0}^{t=T} dt \nabla_{x_t} \cdot f_\theta(x_t, t)$$

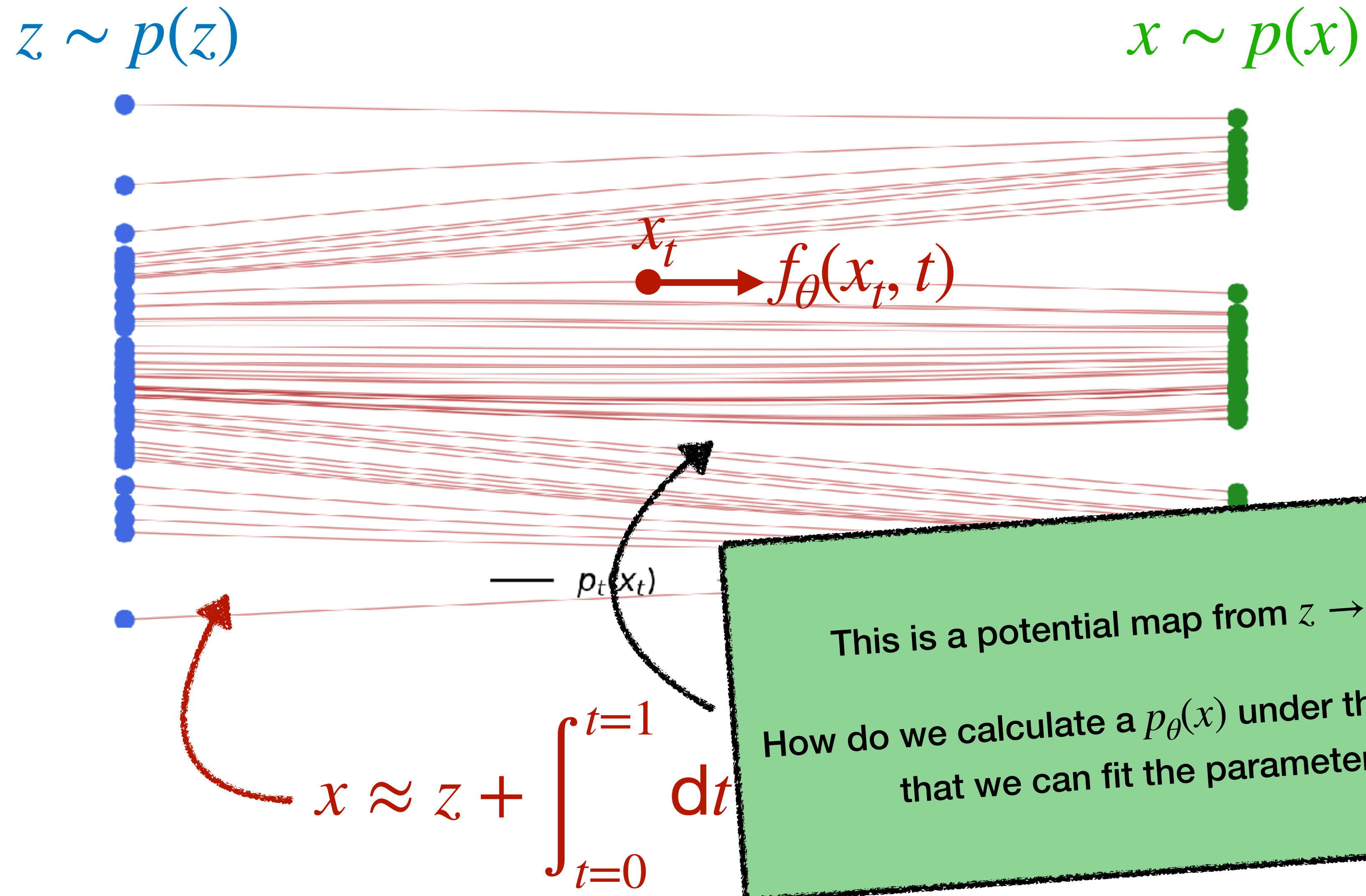
# Continuous Normalising Flows: in practice



# Continuous Normalising Flows: in practice



# Continuous Normalising Flows: in practice

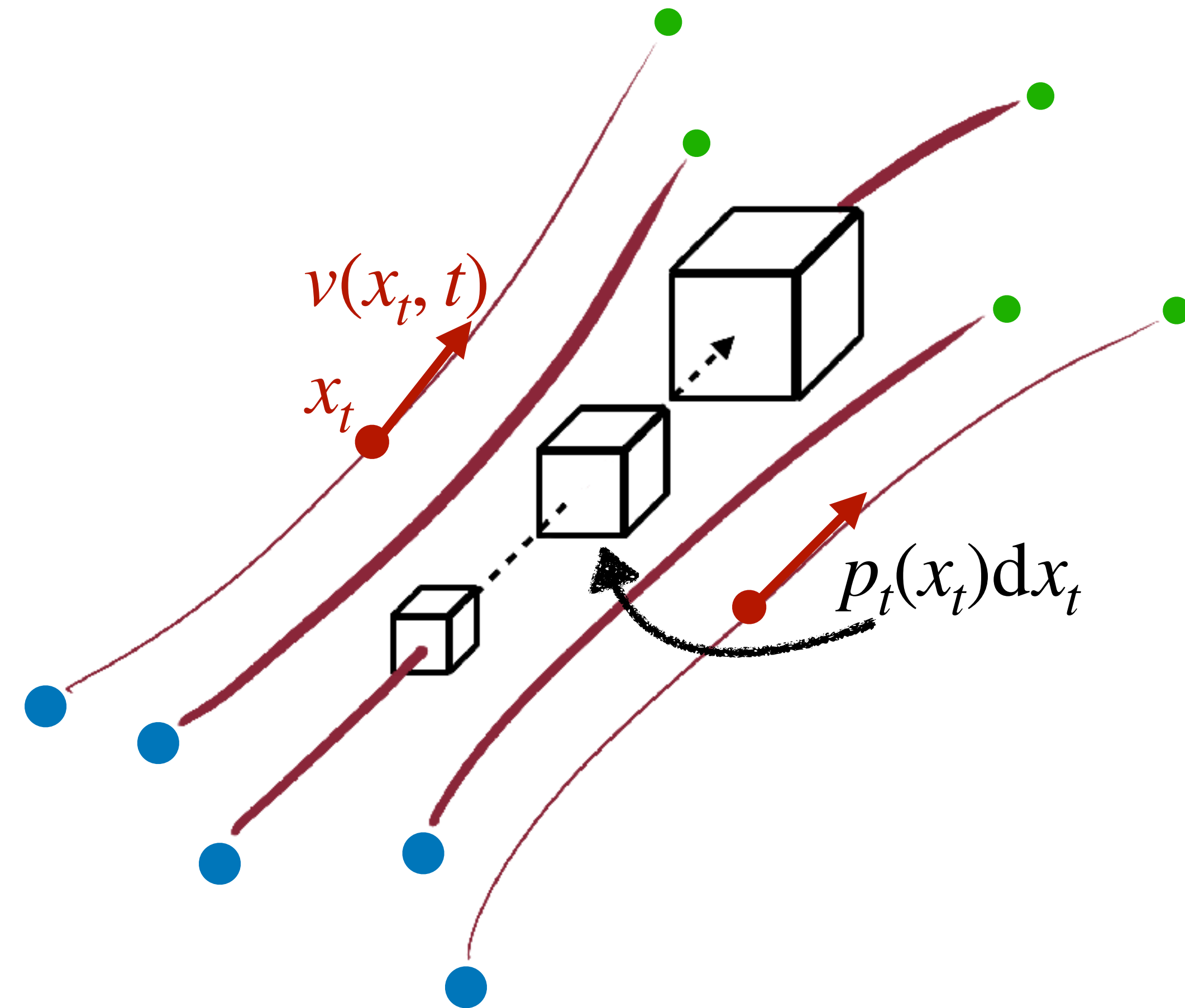


# Continuity equation for probability mass

- We need an equation that describes how probability "flows" at time  $t$ , along a path  $z \rightarrow x$

→ continuity equation for *density*  $p_t(x_t)$

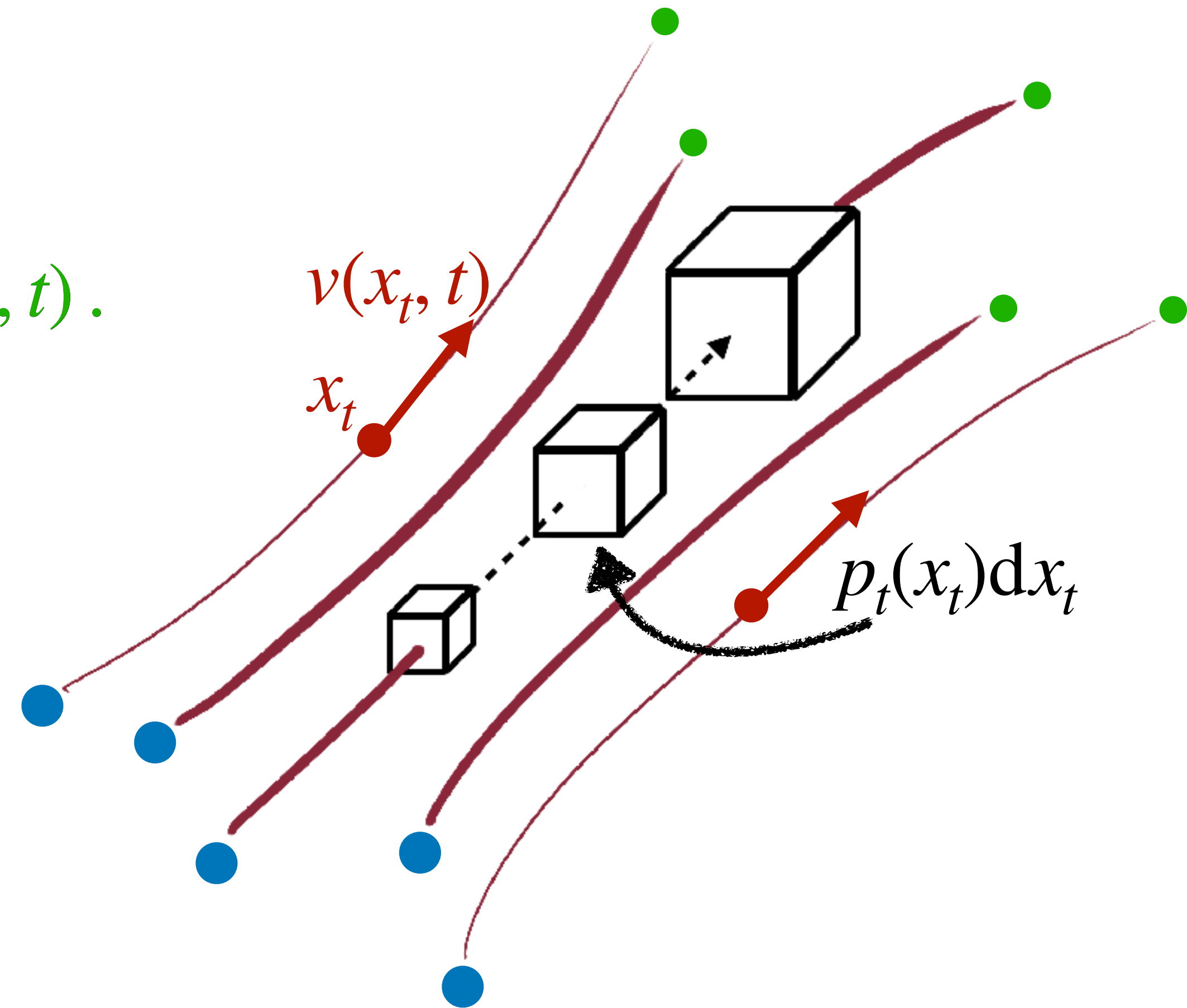
$$\frac{\partial p_t(x_t)}{\partial t} + \nabla_{x_t} \cdot [p_t(x_t) \mathbf{v}(x_t, t)] = 0$$



# Continuity equation for probability mass

- Rate of change of density  $p_t(x_t)$  along the trajectory is given by this ODE

$$\frac{d \log p_t(x_t)}{dt} = - \nabla_{x_t} \cdot v(x_t, t) = - \nabla_{x_t} \cdot f_\theta(x_t, t).$$



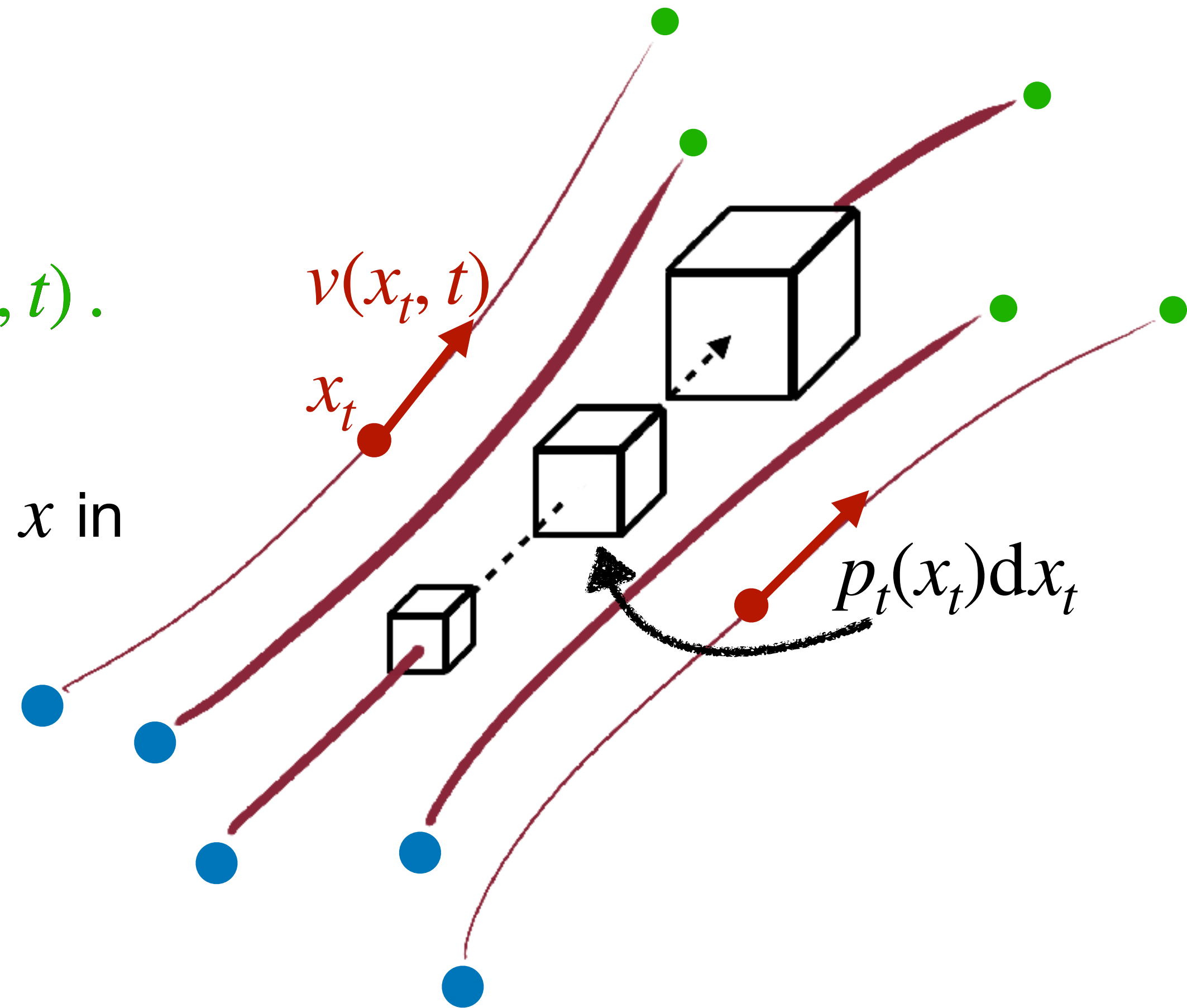
# Continuity equation for probability mass

- Rate of change of density  $p_t(x_t)$  along the trajectory is given by this ODE

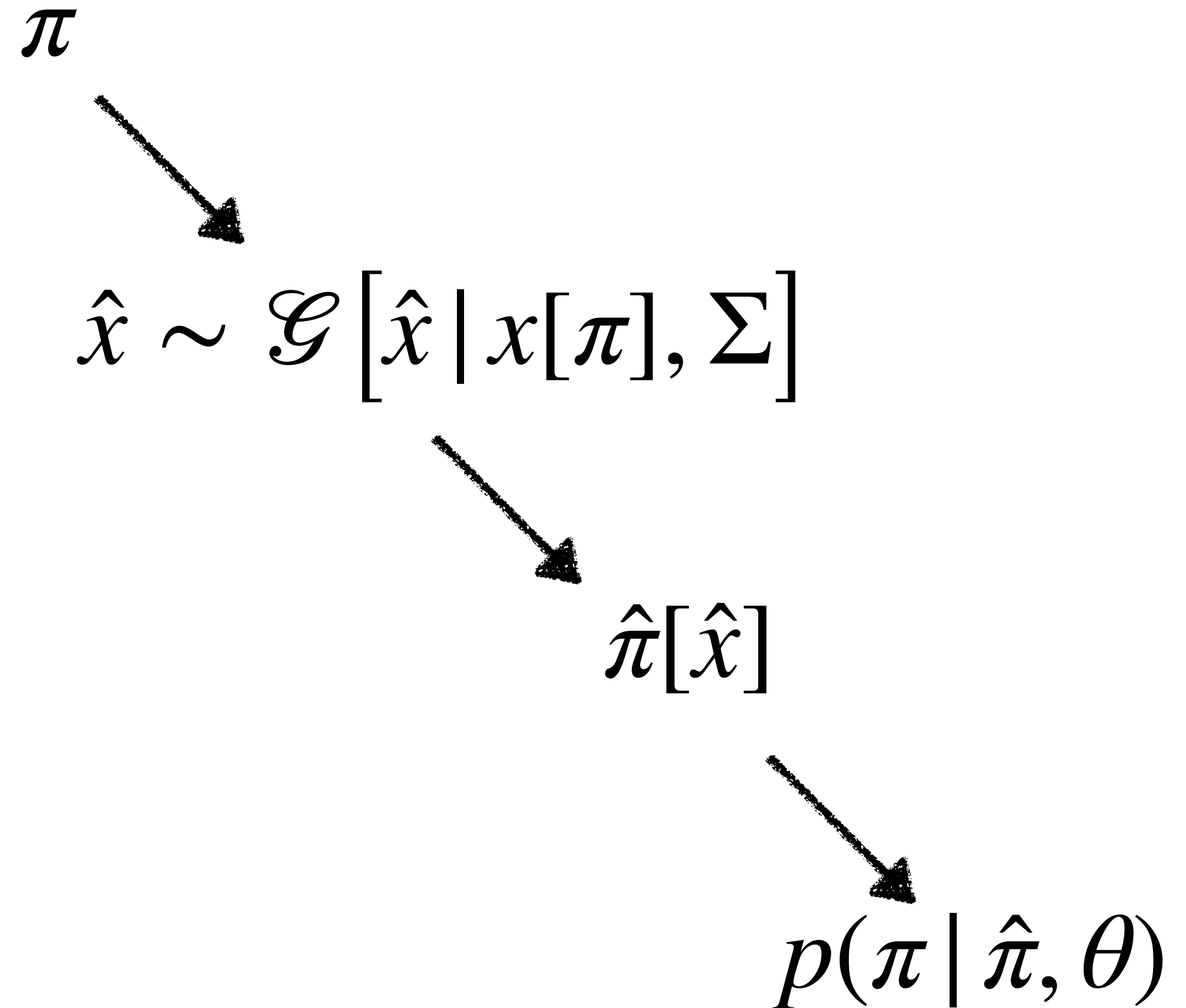
$$\frac{d \log p_t(x_t)}{dt} = - \nabla_{x_t} \cdot v(x_t, t) = - \nabla_{x_t} \cdot f_\theta(x_t, t).$$

- Maximise the integral of this over the path from  $z$  to  $x$  in our training set

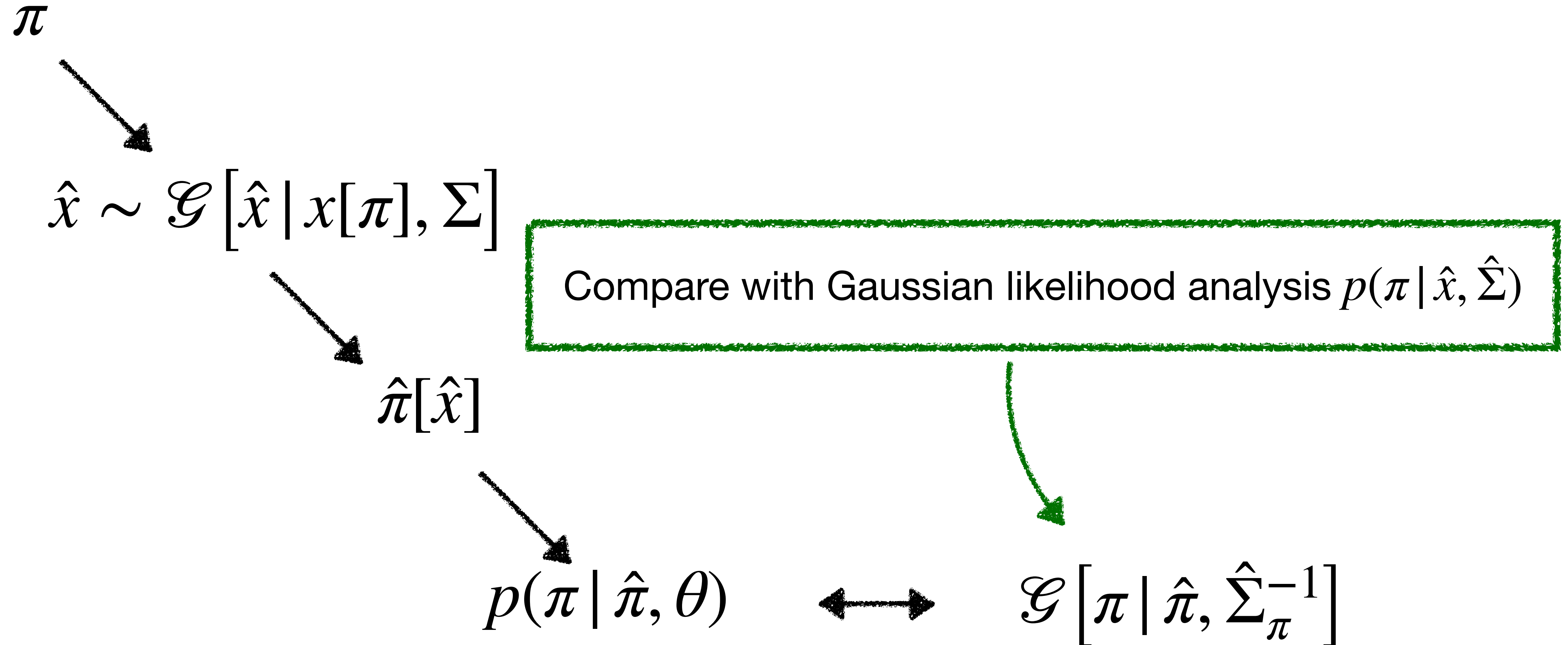
$$\log p_\theta(x) = \log p(z) - \int_{t=0}^{t=1} dt \nabla_{x_t} \cdot f_\theta(x_t, t)$$



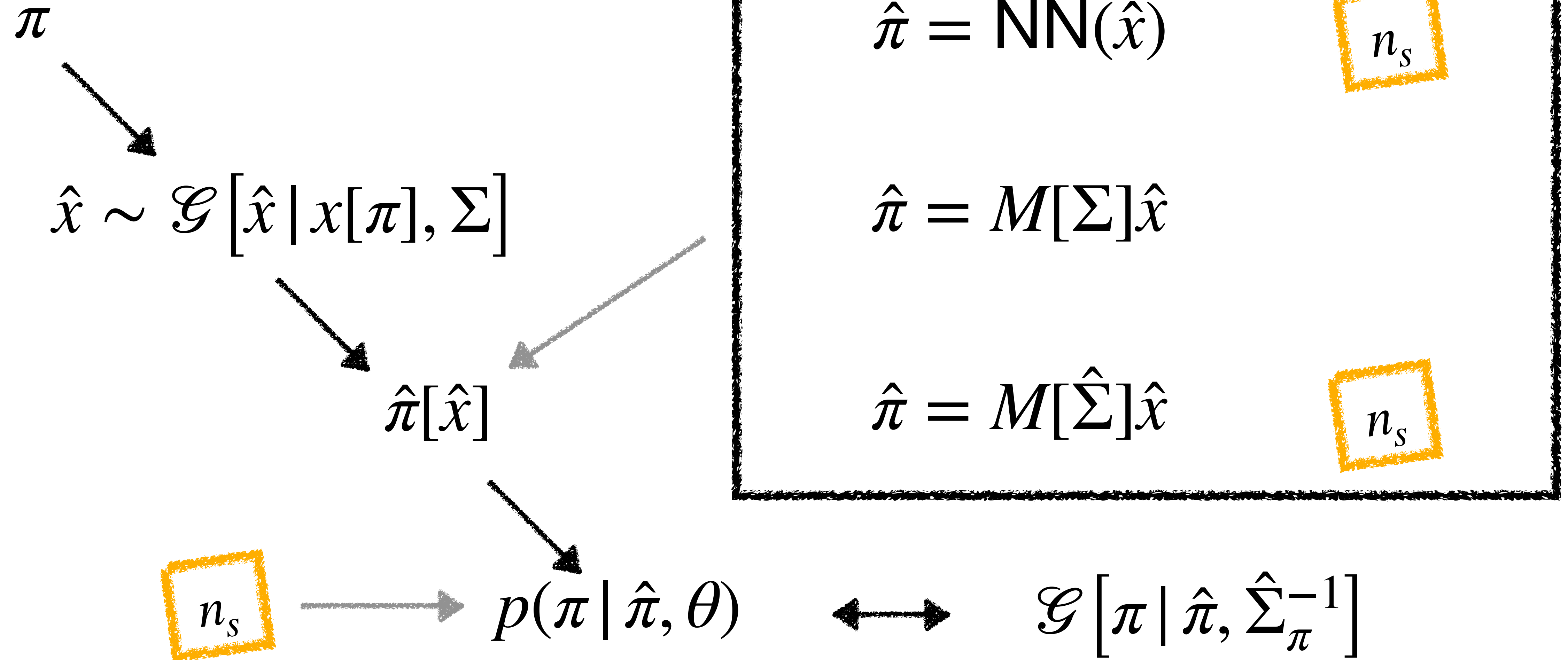
# Analytic testing of SBI



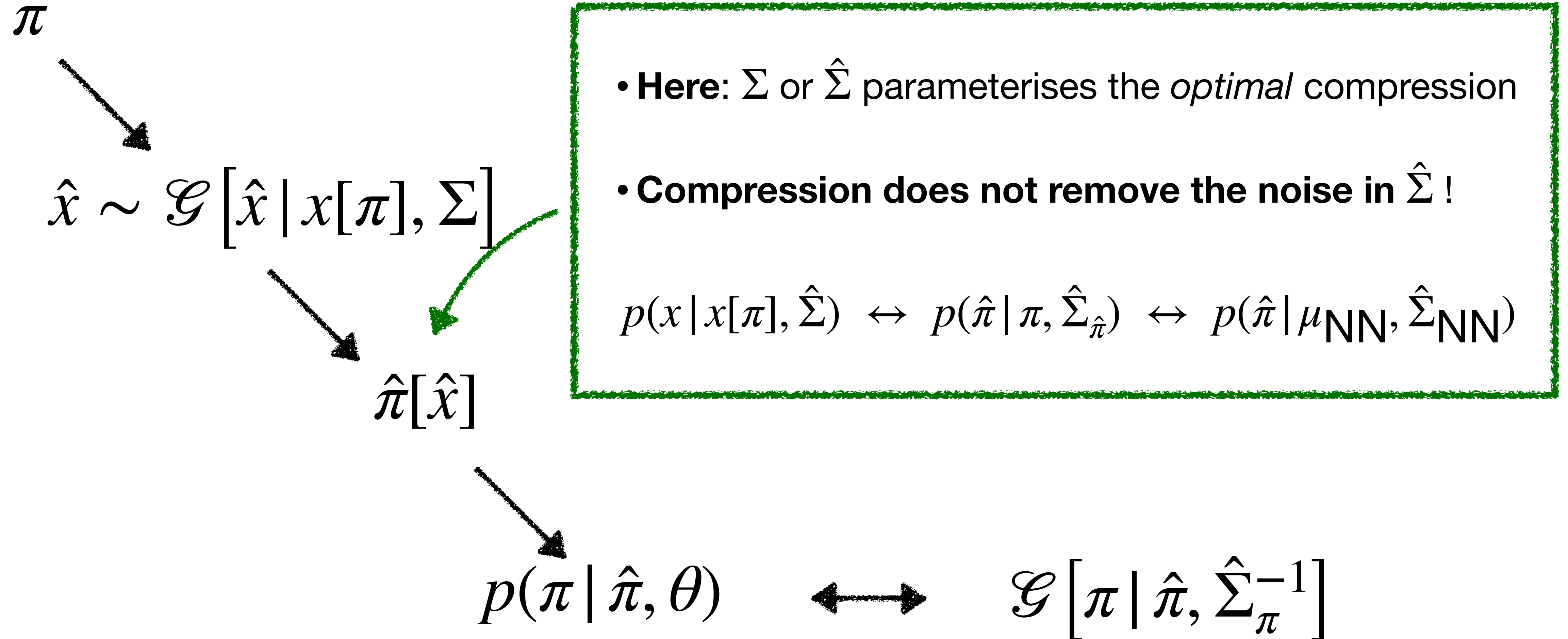
# Analytic testing of SBI



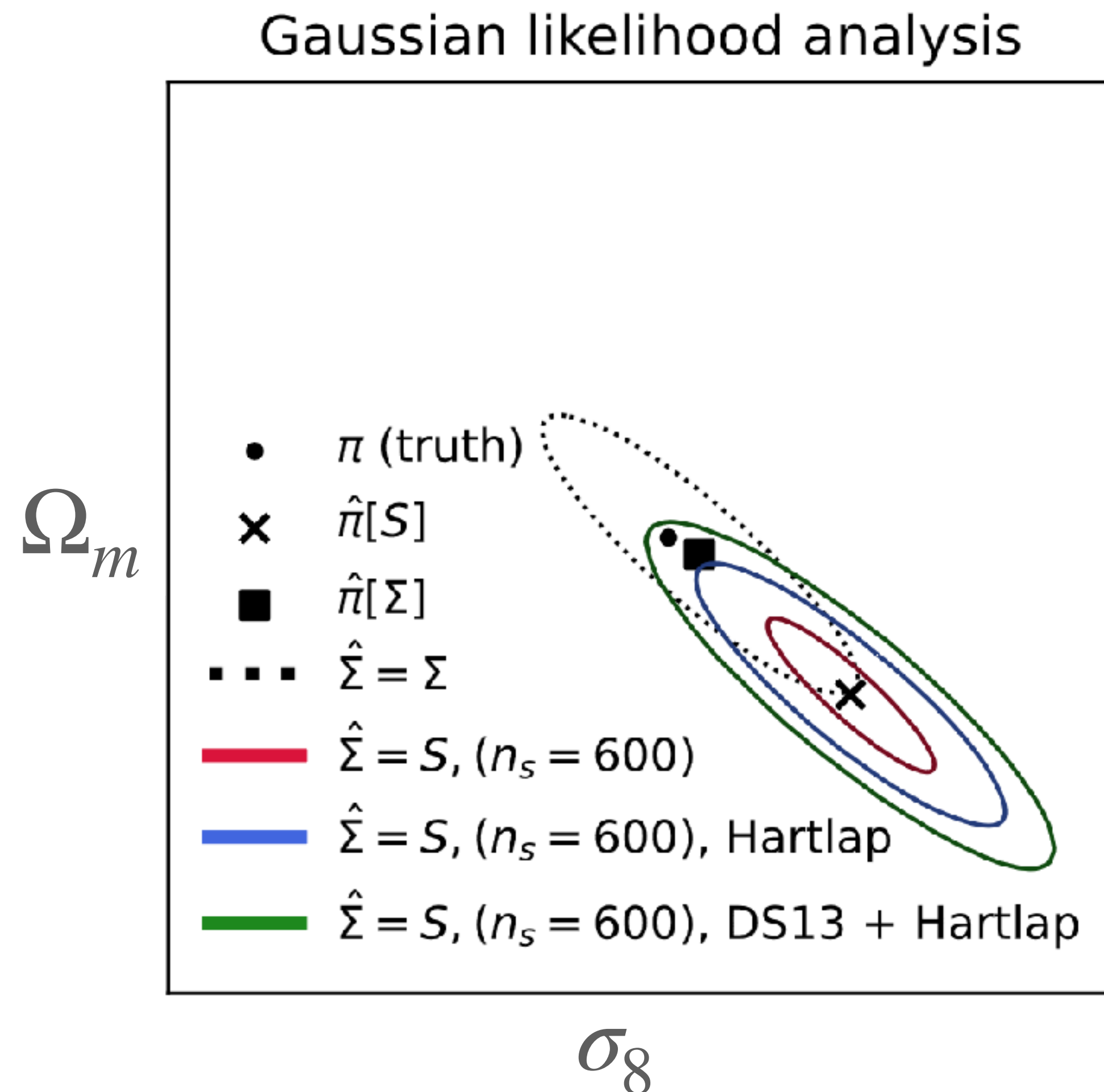
# Analytic testing of SBI



# Analytic testing of SBI

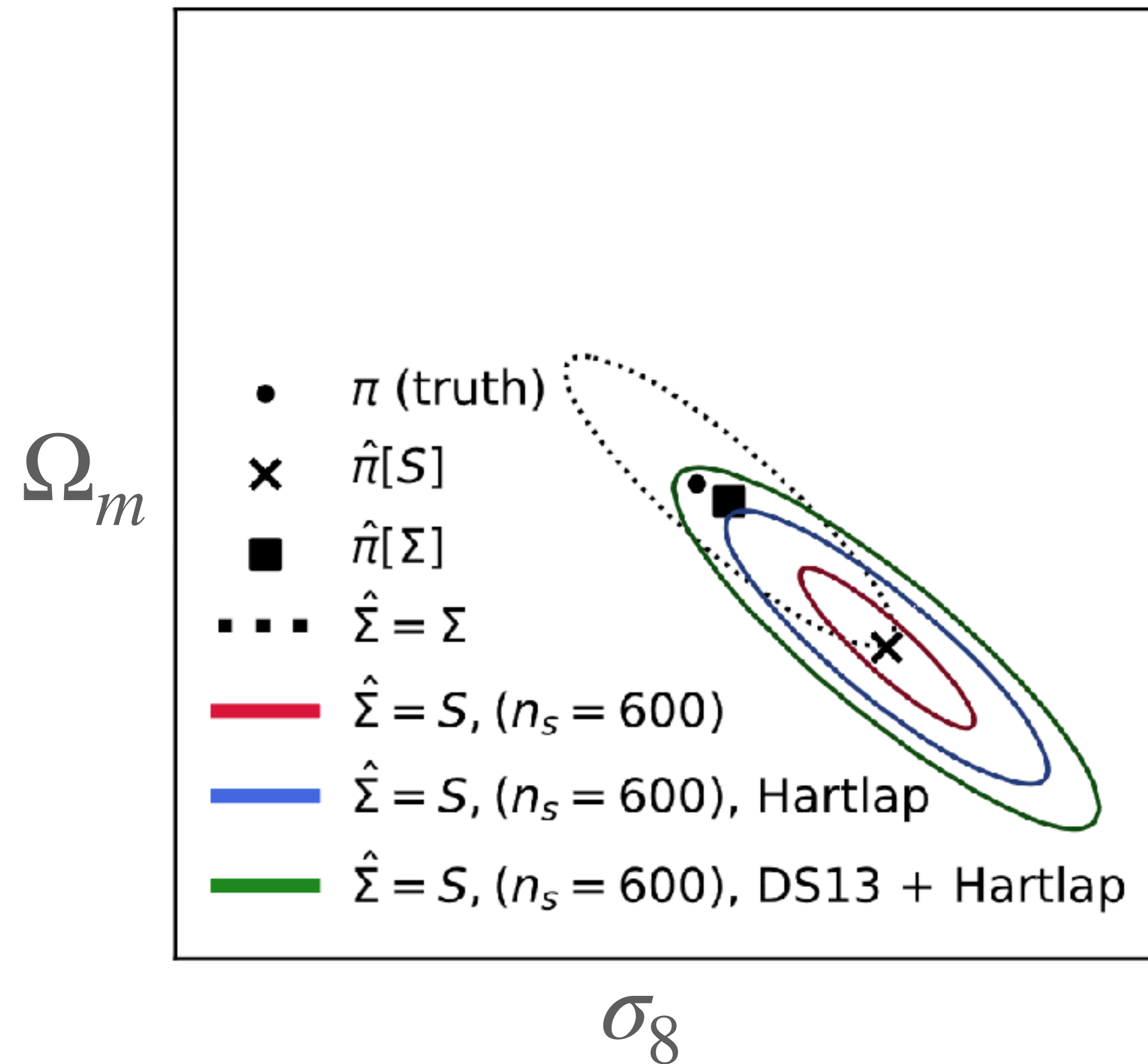


# The good and bad news about SBI

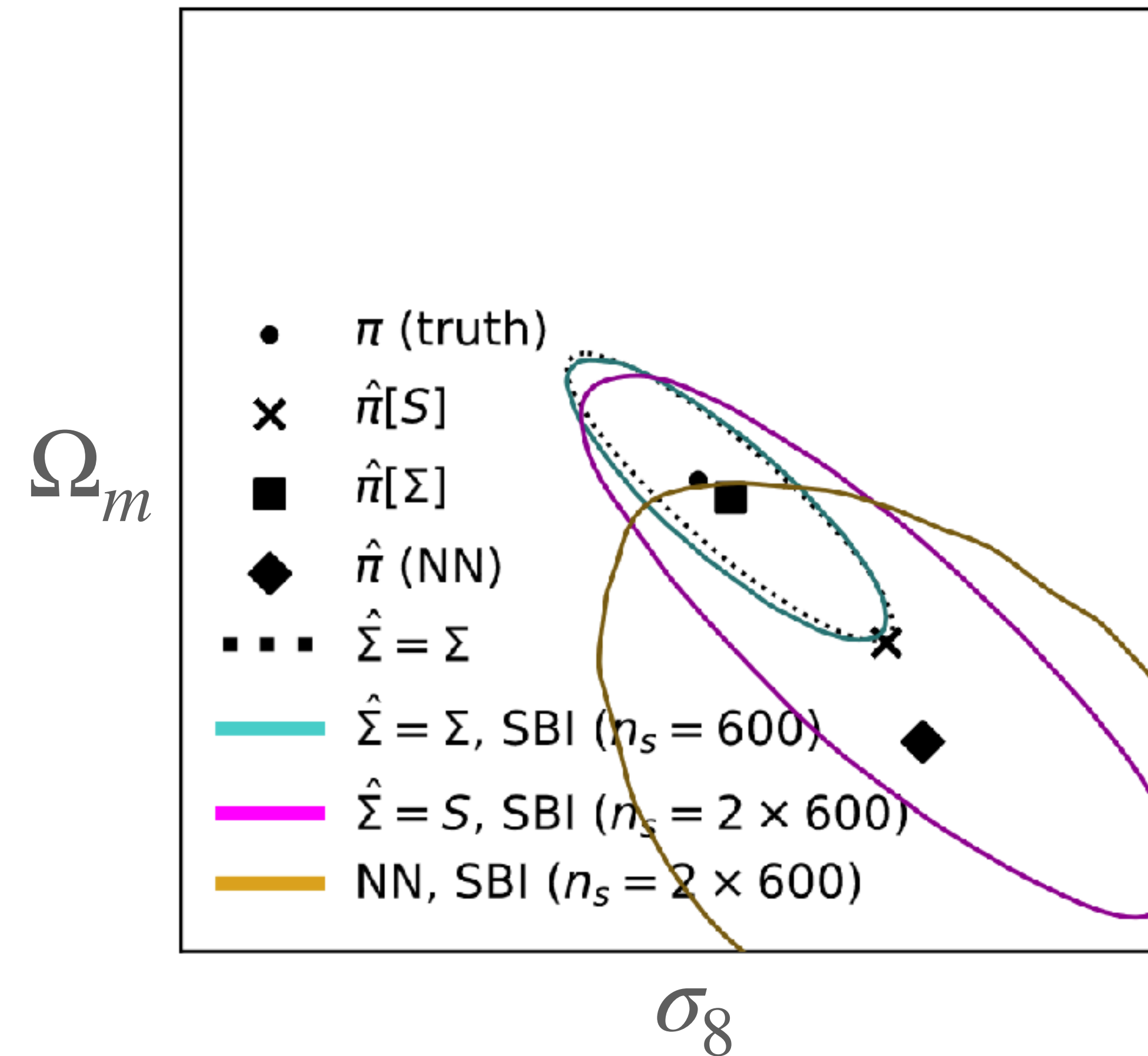


# The good and bad news about SBI

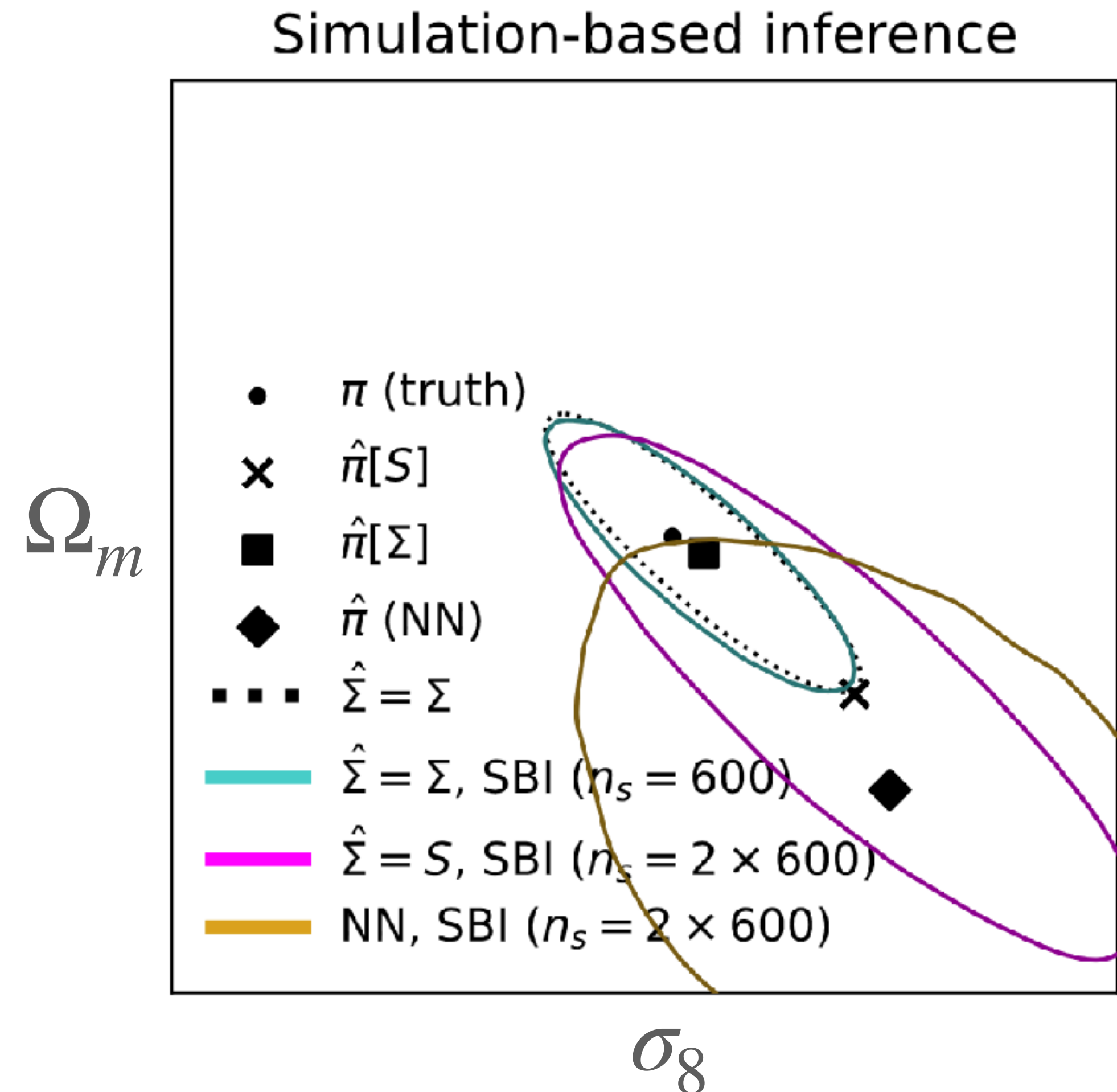
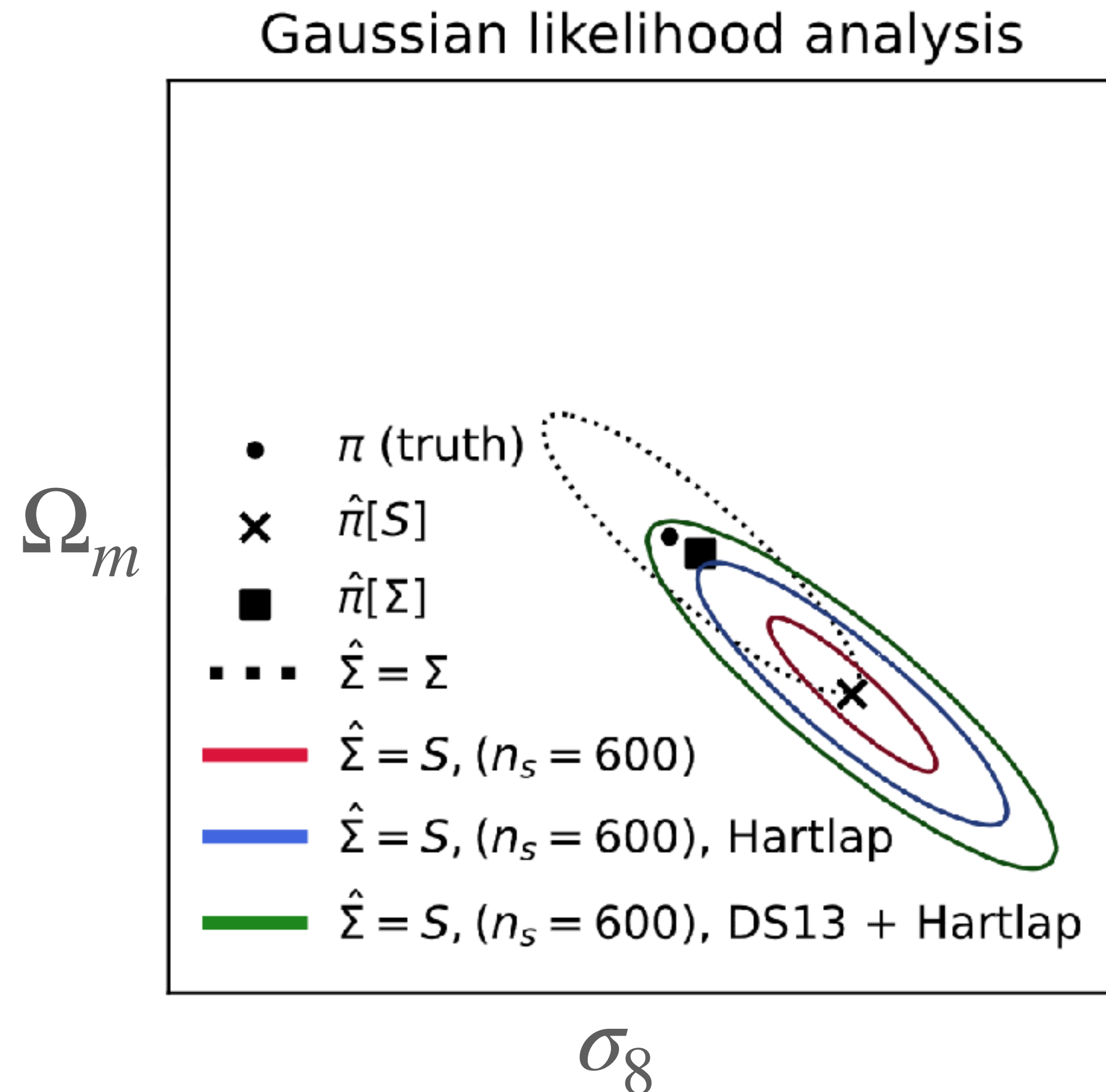
Gaussian likelihood analysis



Simulation-based inference

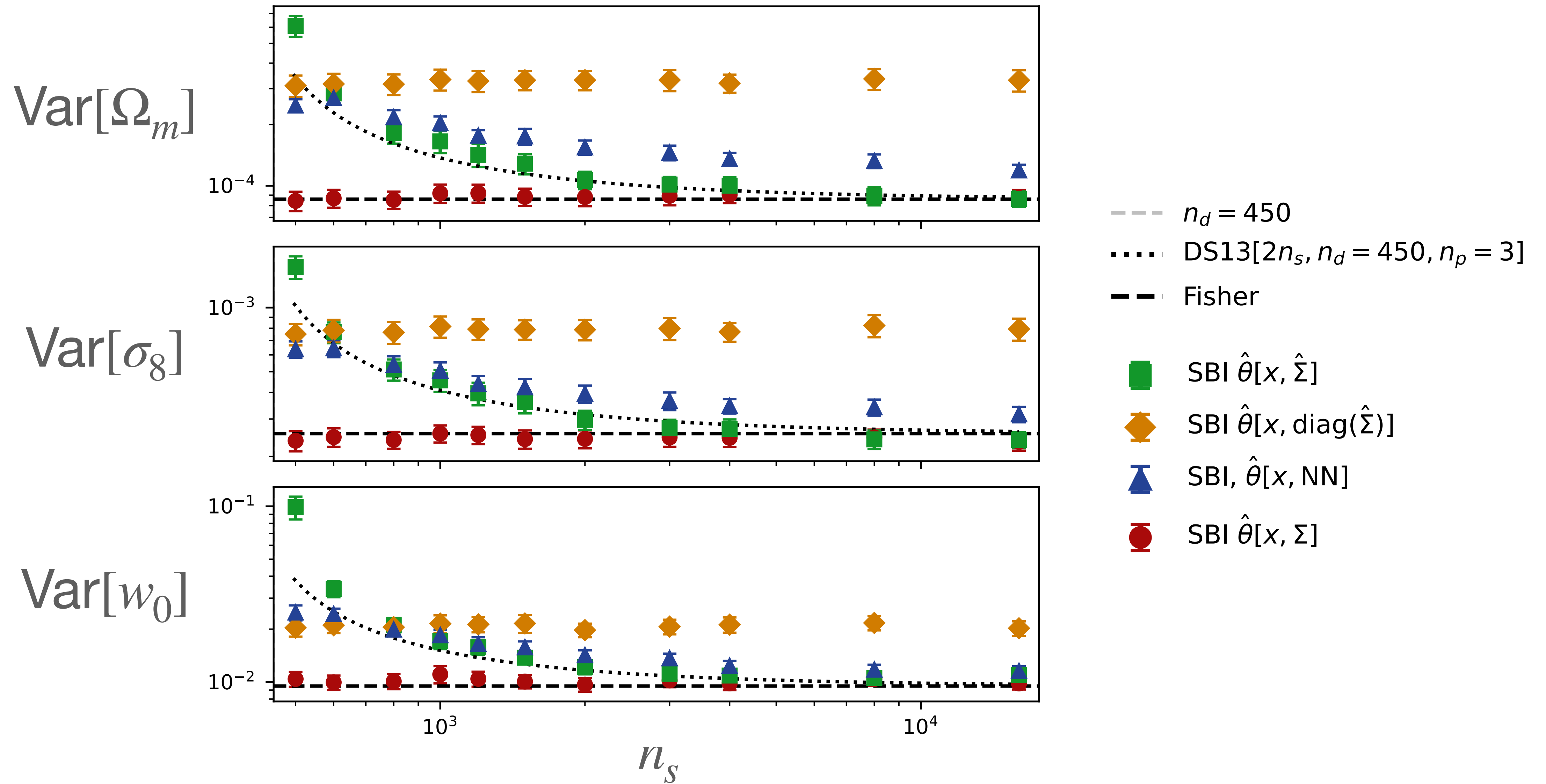


# The good and bad news about SBI

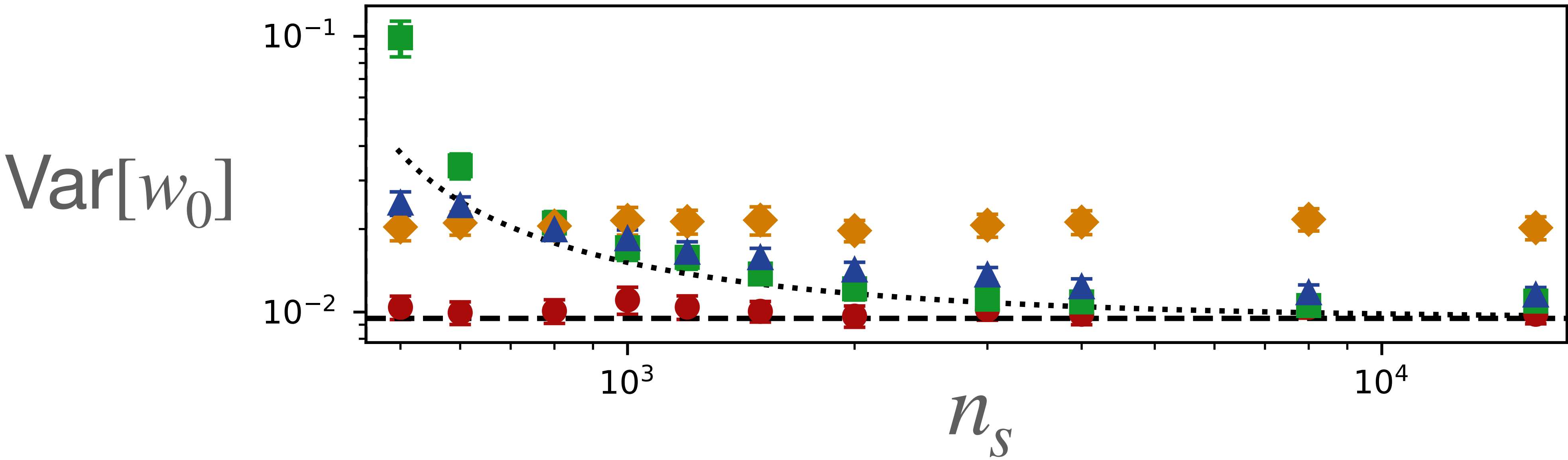


SBI is aware of the Dodelson-Schneider effect but it is inefficient in its response!

# Posterior widths: SBI vs. Gaussian likelihood analysis



# Posterior widths: SBI vs. Gaussian likelihood analysis



---  $n_d = 450$

.... DS13[ $2n_s, n_d = 450, n_p = 3$ ]

--- Fisher

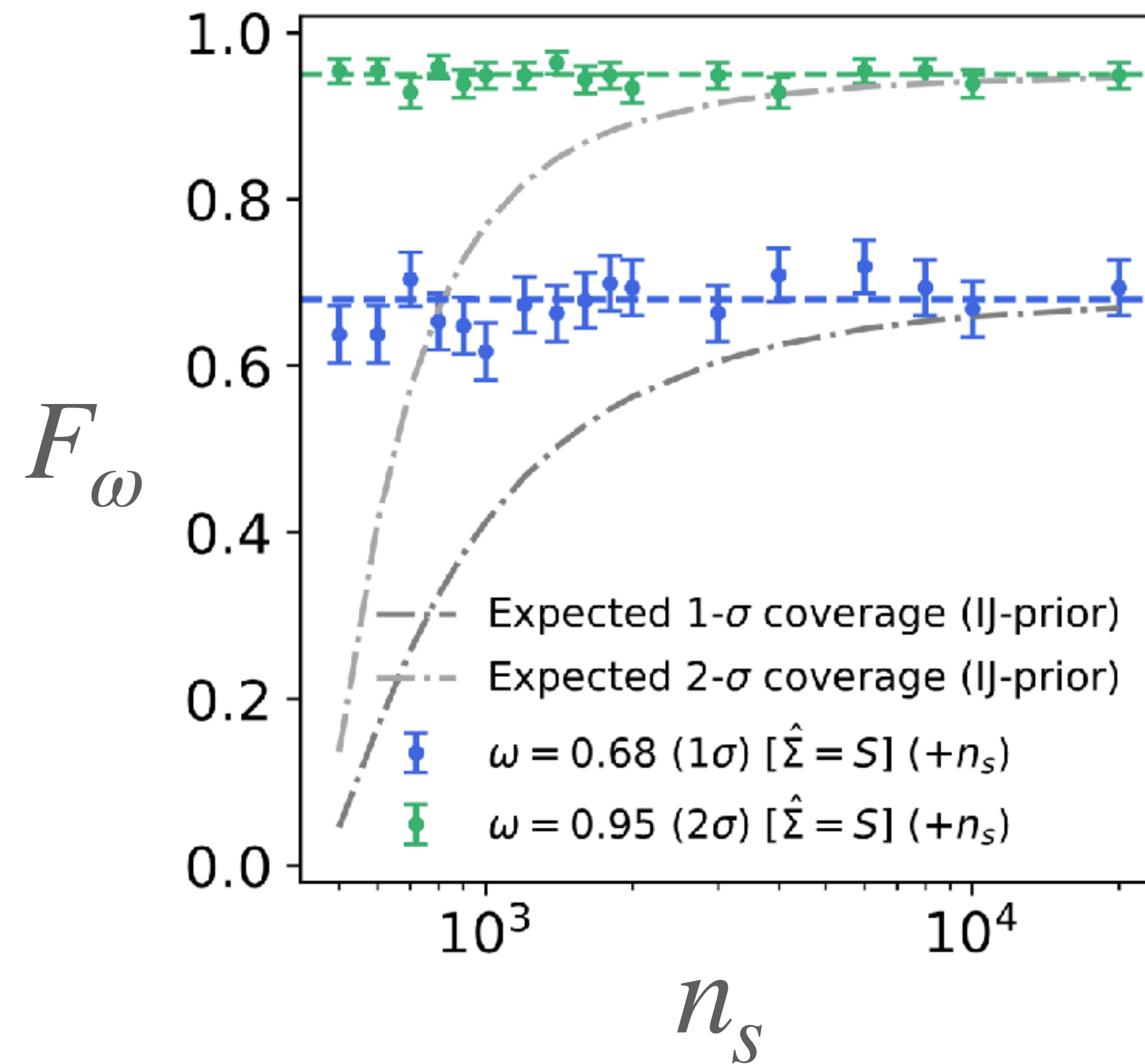
 SBI  $\hat{\theta}[x, \hat{\Sigma}]$

 SBI  $\hat{\theta}[x, \text{diag}(\hat{\Sigma})]$

 SBI,  $\hat{\theta}[x, \text{NN}]$

 SBI  $\hat{\theta}[x, \Sigma]$

# Posterior coverages: SBI



# Good news

- Coverages are *nominal*, widths are *inflated*  $\implies$  **SBI knows about the Dodelson-Schneider effect and corrects for it!**
- **Low  $n_s$  with high  $n_x \implies$  NN + NDE returns tighter constraints** (than  $\mathcal{G} + \hat{\Sigma}$ ), where  $\Sigma$  is insurmountable (analytically or through simulations)
- **No assumptions on  $p(x \mid \pi)$  and optimal compression  $\implies$  SBI fits true  $p(x \mid \pi)$  with low  $n_s$**

# Bad news

- **SBI returns diluted parameter constraints w.r.t. a Gaussian likelihood analysis for the same  $n_s$  for Stage-III surveys**
- **Compression is the crux:** we don't know how to compress data optimally, and multi-probe analyses increase  $n_x$ 
  - **Moving noise in  $\hat{\Sigma}$  around the analysis:** choose either  $p(x | x[\theta], \hat{\Sigma})$ ,  $p(\hat{\theta} | \theta, \hat{\Sigma}_{\hat{\theta}})$ ,  $p(\hat{\theta} | \mu_{\text{NN}}, \hat{\Sigma}_{\text{NN}})$ 
    - **the neural density estimator likelihood fit cannot extract information that is lost here!**
    - **DS13 can obscure non-Gaussian information gain!**
- **Nuisances and complexity in  $x[\theta]$ , parameter dependence of  $\Sigma$  and non-Gaussian  $p(x | \theta)$  will dilute information further, it is unclear how SBI responds to this...**
- **Hyperparameter optimisation:** when  $n_x$  is large, training set is not as large as we'd like, meaning we overfit more to this during hyperparameter optimisation!

Write down something you learned today and  
something you need to hear more about next lecture

 [jed.homer@physik.lmu.de](mailto:jed.homer@physik.lmu.de)